Mímir: A Stateless Private Search Engine

Gökberk Erdoğan*
Technical University of Munich

Emma Munisamy*
Technical University of Munich

Kamil Kluczniak
Independent Cryptographer

Abstract

Private search engines allow a user to retrieve documents from a public database that are similar to a text query. Our cryptographic protocol Mímir employs state-of-the-art homomorphic ciphertext switching techniques to realize the first stateless semantic private search engine for highly dynamic databases. The security of Mímir relies on (Ring) Learning with Errors. Mímir requires no offline communication, and it trades higher but highly parallelizable server computation for low per-query communication. When the database may change between queries, Mímir matches Tiptoe's search quality with 84× less per-query communication (0.24 MiB). At Tiptoe's per-query communication, it reaches 64% instead of 36% of non-private search quality and is 4.2× faster (0.87 s on a single multithreaded machine). Against PACMANN, Mímir improves search quality from 76% to 81% of non-private with 72% of its per-query communication and a 9.54 MiB instead of 2.68 GiB one-time cost, but is 2.1× slower. Mímir can achieve 59% of the non-private search quality with 8.13 MiB of per-query communication and 0.77 s of total latency. This matches PACMANN's latency at a 7.6× smaller per-query communication, and outperforms Tiptoe in all three metrics.

1 Introduction

Modern applications of text search rely on searching not only for keywords, but also for meaning. Web search engines [34], enterprise document retrieval [68] and retrieval-augmented generation (RAG) for large language models [44] all depend on finding text that is semantically related to a query. The dominant method called *semantic search* performs this via vector embeddings [56, 65, 74]. Using a machine learning model trained for this task, this method converts any text string into a fixed-length embedding vector based on its content. Strings that are similar in meaning produce vectors that are close to each other in the embedding space.

*Equal contribution.

While semantic search returns documents with related content for a given query, running it on an untrusted server introduces privacy risks. The query leaks information about what a client is searching for. Alternatively, the client can compute the query embedding locally and transmit only the embedding to the server. But the embeddings naturally leak information about the semantic area of the input [71], and can be inverted to reconstruct the input with high confidence [46, 57].

There are numerous techniques to provide privacy for the client. Anonymizing networks like Tor [27] and VPNs [33] hide the identity and/or the location of the client. Trusted execution environments (TEEs) and hardware enclaves perform the search within a hardware-isolated environment [17, 63], hiding the computation from the operator. Some approaches obfuscate the query by injecting synthetic queries as noise alongside the real one [62]. Although all of these approaches provide some form of protection, none of them performs the search without seeing the query in plaintext.

A notion with stronger privacy guarantees is *private search*, where for any two queries the client makes, the server's view of the interaction is computationally indistinguishable. Private search can be based on cryptographic assumptions: the client encrypts its query, the server computes the search on the encrypted query and obtains the results in encrypted form. Only the client can decrypt these to retrieve the search-relevant documents. Concretely, a private search protocol must (i) preserve the privacy of the query against the server, (ii) match the search quality of semantic search performed over cleartext queries, and (iii) incur efficient communication.

A few private search protocols exist. Coeus [2] ranks the documents according to the tf-idf metric. Tiptoe [36] improves on the quality of Coeus by doing semantic search under encryption, and it proposes clustering the documents according to semantic similarity. Returning the results only for the most relevant cluster cuts down the communication cost at the expense of reduced search quality. The downside of Tiptoe is that for each query to be performed, Tiptoe needs to generate a large one-time token and transmit it to the client in advance, in order to reduce the online latency. Finally, there

is PACMANN [81], which replaces the *semantic search with clustering* idea by a state-of-the-art graph-based search algorithm. This leads to much higher search quality, but incurs larger communication.

All of these protocols rely on a preprocessing step that depends on the database. This is costly in communication and computation, but can be done in a non-latency-critical phase, not affecting the time a client has to wait for its search results. This requires the assumption that the database is static. But in most real-world applications, the search is done over highly dynamic databases, where the client wants to obtain the most up-to-date information related to its search. Anytime a document gets added to or removed from the set, or anytime a document in the pool gets updated, the preprocessing needs to be done again so that querying may continue. In this case, preprocessing cannot be offloaded to a non-latency-critical path.

This paper presents Mímir, a private search engine designed for exactly this setting. Mímir has neither database-dependent preprocessing nor per-query tokens, making it the first semantic private search protocol that handles queries fully online.

At a high level, Mímir preserves the two-round structure of prior work. The first round identifies the most relevant documents for the client's search query, and the second round fetches those documents using an off-the-shelf Private Information Retrieval (PIR) protocol [19, 43]. Mímir focuses on the first round, reducing communication cost and improving search quality while eliminating the need for preprocessing. The second round is unchanged from prior work and can be instantiated with any off-the-shelf stateless PIR scheme.

To find the indices of the most relevant documents for the client's search, Mímir generates semantic vector embeddings for all documents and groups them in clusters [36]. The client generates its query embedding, and encrypts the query embedding and the index of the relevant cluster for which it wants to obtain the similarity scores. Under encryption, the server computes and returns the similarity scores between the query and the documents in the relevant cluster.

Unlike Tiptoe, which encodes the query embedding and the cluster selection jointly as a single padded vector, Mímir separates these into two independent encryptions, achieving smaller communication. Mímir allows trading communication cost for better search quality and latency by adjusting the cluster size. The underlying cryptographic mechanisms neither require a preprocessed database snapshot nor a disposable per-query token.

Below is a list of Mímir's major technical contributions.

No preprocessing. Mímir performs all private search computation online: no huge offline costs are incurred before querying may begin. The client once generates two scheme-switching keys that are independent of the database. These never need to be refreshed, and like ordinary public keys, can even be reused across different servers.

Low online communication. Mímir reduces online communication by encrypting the query embedding and the cluster selection separately, each under a lightweight linear homomorphic encryption scheme. The uniformly random parts of these ciphertexts are transmitted as a small seed to be expanded on the server side, further reducing communication (§3.2.1).

Linear computation time. By storing the embedding database in the NTT domain, Mímir performs the computations in linear time in the database size, which is optimal without preprocessing [9]. For this, the server only needs to transform the query embedding and the cluster selection encryptions into the NTT domain (§3.2.2).

Flexible plaintext packing. Using the Chinese Remainder Theorem, Mímir decomposes the plaintexts into their Residue Number System representation over small component rings. The small component rings allow Mímir to use smaller parameters. Alternatively, for the same parameters, Mímir can use embedding vectors with a higher resolution. This allows Mímir to find the best trade-off between search quality, communication, and latency (§3.2.3).

We implement Mímir and evaluate it over the MS MARCO dataset, measuring search quality via the Mean Reciprocal Rank (MRR@100, higher is better) and latency on a single multithreaded machine. To match each baseline's operating point, we trade between search quality, communication and latency by adjusting the number of clusters. In a dynamic setting where the database may change between queries, Mímir matches Tiptoe's search quality with 0.24 MiB per query instead of 20.26 MiB (84× smaller), at 4.23 s of total latency instead of 3.65 s. At Tiptoe's per-query communication, it reaches 64% instead of 36% of non-private search quality and is 4.2× faster (0.87 s). Against PACMANN, it reaches 81% instead of 76% with 72% of PACMANN's per-query communication and a 9.54 MiB instead of 2.68 GiB one-time upload for the two scheme-switching keys, but takes 1.66 s instead of 0.80 s. Staying within PACMANN's latency budget, Mímir achieves 59% of the non-private search quality with 7.6× less communication (8.13 MiB). This also outperforms Tiptoe on all three performance metrics (§4.2).

2 Preliminaries

Notation. We denote the set of natural numbers, including zero, by $\mathbb{N}$, the set of positive integers by $\mathbb{N}_+ := \mathbb{N} \setminus \{0\}$, and the set of integers by $\mathbb{Z}$. For any positive integer q, $\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z}$ denotes integers modulo q. For a positive power-of-two integer N, we denote the polynomial ring $\mathbb{Z}_q[X]/(X^N+1)$ by $\mathcal{R}_q$. The operator $\xleftarrow{\mathcal{D}} \mathcal{S}$ denotes sampling randomly from a set $\mathcal{S}$ according to a distribution $\mathcal{D}$; the special case of sampling uniformly at random is denoted as $\xleftarrow{R}$. Logarithms are base 2 unless specified otherwise. We denote vectors with bold lowercase letters, and matrices with bold uppercase letters. For a vector $\mathbf{v}$, $\|\mathbf{v}\|$ denotes its ℓ_2-norm. The operator $\lfloor \mathbf{v} \rceil$ rounds each entry of $\mathbf{v}$ to the nearest integer. The ith entry of a vector $\mathbf{v}$ is denoted by $\mathbf{v}[i]$. The symbol $\mathbf{u}_i$ denotes the unit

vector with a 1 at the ith position. The notation $[k]$ represents the integer set $\{0,1,\ldots,k-1\}$. We define a polynomial over a ring $\mathcal{R}_q$ as $\mathfrak{v} := \sum_{i=0}^{N-1} \mathbf{v}[i] \cdot X^i$, and denote its coefficient vector by $\text{Coefs}(\mathfrak{v}) = \mathbf{v}$.

Similarity measures. We consider two metrics to measure the *similarity* between two embedding vectors. Let $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_p^l$ for $p \geq 2$. The *inner product* $\langle \mathbf{x}, \mathbf{y} \rangle$ and the *cosine similarity* $\cos(\mathbf{x}, \mathbf{y})$ are

$$\langle \mathbf{x}, \mathbf{y} \rangle := \sum_{i=1}^{l} \mathbf{x}[i] \cdot \mathbf{y}[i] \quad \text{and} \quad \cos(\mathbf{x}, \mathbf{y}) := \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \cdot \|\mathbf{y}\|},$$

where $\cos(\mathbf{x}, \mathbf{y}) \in [-1, 1]$.

2.1 Private Search

In private search, a client retrieves information relevant to its query from a server-held database without revealing the query, the relevant documents, or – in clustered settings – the query-relevant cluster to the server.

Private search protocols commonly follow the two-step framework [2, 36, 81] shown in Fig. 1: the first step privately determines which documents are most relevant to a given query, and the second step fetches them from the server via PIR. The server accordingly maintains two data structures: an index for identifying relevant documents, and the document store itself. For semantic private search, the former is an *embedding database*, containing a semantic embedding vector for every document, computed once during setup.

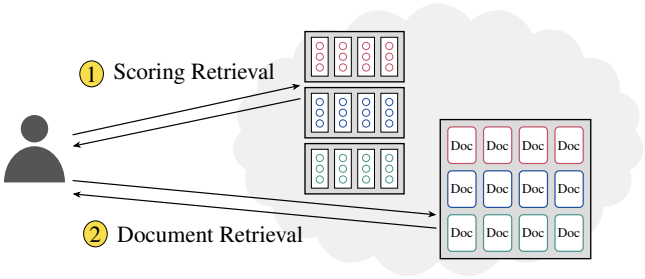


Figure 1: The two steps of private search.

We refer to the first step as *scoring retrieval*. Here, the client computes the query embedding, encrypts it, and sends it to the server. The server then homomorphically evaluates the inner-product or cosine similarity between the encrypted query embedding and each embedding in the database, yielding an encrypted *similarity score* for each document. We refer to the resulting vector of encrypted scores as the *scoring vector*.

We refer to the second step as *document retrieval*. In this step, the client uses a single-server PIR protocol (§2.3) to retrieve the documents relevant to its query. If multiple documents are to be retrieved, batch-PIR can be used instead.

We write a private search protocol as the composition of these two steps, shown in Algorithm 1. ScoringRetrieval takes the document set, a machine learning model that generates vector embeddings from variable-sized text-string inputs, and a client query as input. It returns a scoring vector consisting of the document scores for the given query. The client then picks the index with the highest score and retrieves the corresponding document via PrivateInformationRetrieval (§2.3).

Algorithm 1: PrivateSearch$(\mathcal{D}, \mathsf{M}, \text{query})$

Input: a set of documents $\mathcal{D} = (\text{doc}_i)_{i \in [D]}$ held by the server, a pretrained model M, a client query
Output: the document $\text{doc}_{i'}$ most relevant to query

$\mathbf{r} \leftarrow \text{ScoringRetrieval}(\mathcal{D}, \mathsf{M}, \text{query})$;
$i' \leftarrow \text{argmax}_i \, \mathbf{r}[i]$;
$\text{doc}_{i'} \leftarrow \text{PrivateInformationRetrieval}(\mathcal{D}, i')$;
return $\text{doc}_{i'}$

This private search framework uses embedding vectors and measures similarity via inner product or cosine similarity. Beyond that, the framework has no other restrictions: the embedding model, length, precision, and the type of search (text, image, audio [36]) can be chosen freely. One technique to reduce communication cost is partitioning the embedding database into clusters [36], so that the server returns similarity scores for only the most relevant cluster.

The choice of the embedding model and clustering directly affects search quality. Additionally, clustering has direct consequences on the communication cost and latency. A search with fewer clusters has higher quality and lower latency, but it costs more to communicate. Sophisticated choices for the embedding model and clustering technique improve search quality.

2.2 Computing on Encrypted Data

Linearly homomorphic encryption. A linearly homomorphic encryption (LHE) scheme is a scheme that can perform additions and plaintext multiplications under encryption.

Let $m_i, m_i' \in \mathbb{Z}_p$ (or $\mathcal{R}_p$), $\forall i \in [k]$, be plaintexts, and let $\mathsf{LHE}(\mathsf{sk}, m_i)$ be the encryption of m_i under a linearly homomorphic encryption scheme LHE using the secret key sk. Then, up to a certain number k depending on the scheme's parameters, it holds that

$$\sum_{i=1}^{k} m_i' \cdot \mathsf{LHE}(\mathsf{sk}, m_i) = \mathsf{LHE}\left(\mathsf{sk}, \sum_{i=1}^{k} m_i' \cdot m_i\right), \qquad (1)$$

where $m_i' \cdot \mathsf{LHE}(\mathsf{sk}, m_i)$ is a *plaintext multiplication* and $m_i' \cdot m_i$ is multiplication over the corresponding ring.

Learning with Errors. We briefly recall Learning with Errors (LWE) and the Regev encryption scheme [64], alongside their ring variants [48].

Definition 1 (Generalized Learning with Errors (GLWE))**.** Let $n, N, q \in \mathbb{N}_+$ and let $\mathcal{X}_s, \mathcal{X}_e$ be distributions over $\mathcal{R}_q =$

$\mathbb{Z}_q[X]/(X^N+1)$ with nonzero standard deviation, chosen according to a security parameter λ.

Given that $\mathfrak{a} \xleftarrow{R} \mathcal{R}_q^n, \mathfrak{s} \xleftarrow{\chi_s} \mathcal{R}_q^n, \mathfrak{e} \xleftarrow{\chi_e} \mathcal{R}_q$, the decisional-GLWE problem is to distinguish whether $\mathfrak{b} = -\mathfrak{a}\cdot\mathfrak{s}+\mathfrak{e}$ or $\mathfrak{b} \xleftarrow{R} \mathcal{R}_q$. Formally, we assume for all PPT distinguishers $\mathcal{A}$,

$$\left|\Pr[\mathcal{A}(\mathfrak{a}, -\mathfrak{a}\cdot\mathfrak{s}+\mathfrak{e})] - \Pr[\mathcal{A}(\mathfrak{a}, \mathfrak{b} \xleftarrow{R} \mathcal{R}_q)]\right| \leq \mathrm{negl}(\lambda),$$

where $\mathrm{negl}(\lambda)$ is a negligible function in the security parameter. GLWE reduces to LWE for $N=1$, and to Ring-LWE (RLWE) for $n=1$.

For $p \in \mathbb{N}_+$ denoting the plaintext modulus with $p \leq q$, and $\Delta := \lfloor q/p \rceil$, we can use LWE to encrypt a message $m \in \mathbb{Z}_p$ under a secret key $\mathbf{s} \xleftarrow{\chi_s} \mathbb{Z}_q^n$ as

$$\mathrm{LWE}_{n,q,\sigma}(\mathbf{s}, m) = (\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s}\rangle + e + \Delta m) \in \mathbb{Z}_q^n \times \mathbb{Z}_q,$$

where $e \xleftarrow{\chi_e} \mathbb{Z}_q$ and $\mathcal{X}_e$ denotes the discrete Gaussian distribution over $\mathbb{Z}_q$ with zero mean and standard deviation σ. This can be extended to encrypt a vector $\mathbf{m} \in \mathbb{Z}_p^\ell$:

$$\mathrm{LWE}_{n,q,\sigma}(\mathbf{s}, \mathbf{m}) = (\mathbf{A}, \mathbf{b} = -\mathbf{A}\mathbf{s} + \mathbf{e} + \Delta\mathbf{m}) \in \mathbb{Z}_q^{\ell\times n} \times \mathbb{Z}_q^\ell,$$

where $\mathbf{e} \xleftarrow{\chi_e} \mathbb{Z}_q^\ell$. Here, the encryption of a message vector $\mathbf{m}$ is the row concatenation of LWE encryptions of $(\mathbf{m}[i])_{i\in[\ell]}$. Similarly, we can use RLWE to encrypt a message $\mathfrak{m} \in \mathcal{R}_p$:

$$\mathrm{RLWE}_{N,q,\sigma}(\mathfrak{s}, \mathfrak{m}) = (\mathfrak{a}, \mathfrak{b} = -\mathfrak{a}\cdot\mathfrak{s} + \mathfrak{e} + \Delta\mathfrak{m}) \in \mathcal{R}_q \times \mathcal{R}_q,$$

where $\mathfrak{e} \xleftarrow{\chi_e} \mathcal{R}_q$.

For the encryptions above, decryption succeeds as long as $|e| < \Delta/2$ and $|\mathbf{e}[i]|, |\mathfrak{e}[i]| < \Delta/2$, $\forall i \in [n]$. Notice that these encryptions are linearly homomorphic according to the definition in Equation (1), where each linear operation increases the standard deviation of the error. For simplicity, we omit N, n and q from our LWE and RLWE encryption notation when it is clear from context. We also omit σ for fresh LWE and RLWE samples, but write $\mathrm{LWE}_{\sigma'}$ or $\mathrm{RLWE}_{\sigma'}$ for any $\sigma' \neq \sigma$.

Beyond standard LWE and RLWE encryption, Mímir uses other encryption schemes. In particular, it achieves low communication costs by performing homomorphic multiplications between RLWE ciphertexts and Ring-GSW (RGSW) ciphertexts, which we describe below.

Gadget decomposition [32]. Gadget decomposition is a technique used in homomorphic encryption to control noise growth. For an integer decomposition base $B \geq 2$, an integer $d := \lfloor \log_B q \rfloor + 1$ and an integral vector $\mathbf{g} := (1, B, \ldots, B^{d-1})$, gadget decomposition $\mathbf{g}^{-1} : \mathbb{Z}_q \to \mathbb{Z}^d$ is a map that satisfies $\langle \mathbf{g}^{-1}(a), \mathbf{g}\rangle = a \bmod q$ for all $a \in \mathbb{Z}_q$.

LWE′, RLWE′ and RGSW encryptions [26, 54]. Using gadget decomposition, we define the LWE′ encryption as

$$\mathrm{LWE}'(\mathbf{s}, \mathbf{m}) := (\mathrm{LWE}(\mathbf{s}, B^i\mathbf{m}))_{i\in[d]}.$$

The RLWE′ encryption is defined similarly as

$$\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{m}) := (\mathrm{RLWE}(\mathfrak{s}, B^i\mathfrak{m}))_{i\in[d]}.$$

LWE′ and RLWE′ ciphertexts are respectively concatenations of d-many LWE and RLWE ciphertexts, where each ciphertext encrypts the message multiplied by the corresponding coefficient of the gadget decomposition vector. Finally, we define the RGSW encryption as

$$\mathrm{RGSW}(\mathfrak{s}, \mathfrak{m}) := \left(\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s}\cdot\mathfrak{m}), \mathrm{RLWE}'(\mathfrak{s}, \mathfrak{m})\right). \quad (2)$$

2.3 Cryptographic Machinery

Scheme-switching lets a protocol use the encryption best suited to each step: LWE encryptions are compact for single messages, RLWE encryptions can pack multiple messages, and RGSW encryptions enable homomorphic multiplications. We rely on existing scheme-switching algorithms in a black-box manner to move between these encryptions, and refer the reader to the source papers for further details.

The parameters n and q govern a scheme's security and decryption failure rate (DFR). The decomposition bases B_1 and B_2 trade off noise growth for efficiency: larger decomposition bases reduce communication and computation at the expense of faster noise growth and thus a higher DFR.

LWEtoRLWE – KeyGen [16]

Input: LWE secret key $\mathbf{s} \in \mathbb{Z}_q^n$, decomposition base B_1.
Output: RLWE-switch key $\mathbf{K}_1 \in \mathcal{R}_q^{d_1\times 2}$, where $d_1 = \lfloor \log_{B_1} q \rfloor + 1$.

LWEtoRLWE [16]

Input: LWE ciphertext $\mathrm{LWE}(\mathbf{s}, m) \in \mathbb{Z}_q^{n+1}$, RLWE-switch key $\mathbf{K}_1$.
Output: RLWE ciphertext $\mathrm{RLWE}_{\sigma_R}(\mathfrak{s}, \mathfrak{m}) \in \mathcal{R}_q^2$, where $\mathrm{Coefs}(\mathfrak{s}) = \mathbf{s}$ and $\mathrm{Coefs}(\mathfrak{m}) = (m, 0, \ldots, 0)$.
Description: The algorithm converts an LWE ciphertext encrypting a message m to an RLWE ciphertext encrypting the constant polynomial message $\mathfrak{m}$. The standard deviation of the error increases to σ_R. $\mathrm{Coefs}(\mathfrak{s}) = \mathbf{s}$ implies $n = N$.

As LWE′ and RLWE′ ciphertexts consist of d_2-many LWE and RLWE ciphertexts respectively, one can switch from LWE′ to RLWE′ using LWEtoRLWE d_2 times.

RLWEtoRGSW – KeyGen [26]

Input: RLWE secret key $\mathfrak{s} \in \mathcal{R}_q$, decomposition base B_2.
Output: RGSW-switch key $\mathbf{K}_2 \in \mathcal{R}_q^{d_2\times 2}$, where $d_2 = \lfloor \log_{B_2} q \rfloor + 1$.

LWE′toRGSW [26]

Input: LWE′ ciphertext $\mathrm{LWE}'(\mathfrak{s}, m)$, RLWE-switch key $\mathbf{K}_1$, RGSW-switch key $\mathbf{K}_2$.
Output: RGSW ciphertext $\mathrm{RGSW}_{\sigma_\mathsf{G}}(\mathfrak{s}, \mathfrak{m}) \in \mathcal{R}_q^{d_2 \times 2}$, where $\mathrm{Coefs}(\mathfrak{s}) = \mathbf{s}$ and $\mathrm{Coefs}(\mathfrak{m}) = (m, 0, \ldots, 0)$.
Description: The algorithm converts an LWE′ ciphertext encrypting a message m to an RGSW ciphertext encrypting the constant polynomial message $\mathfrak{m}$. The standard deviation of the error increases to σ_G.

The LWE and RLWE encryptions are only linearly homomorphic. To do homomorphic multiplications, we use the following algorithm.

RLWE×RGSW [26]

Input: RLWE ciphertext $\mathrm{RLWE}(\mathfrak{s}, \mathfrak{m}_1)$, RGSW ciphertext $\mathrm{RGSW}(\mathfrak{s}, \mathfrak{m}_2)$.
Output: RLWE ciphertext $\mathrm{RLWE}_{\sigma_\times}(\mathfrak{s}, \mathfrak{m}_1 \cdot \mathfrak{m}_2)$.
Description: The algorithm multiplies an RLWE ciphertext encrypting a message $\mathfrak{m}_1$ with an RGSW ciphertext encrypting a message $\mathfrak{m}_2$, both of which are encrypted with the same secret key $\mathfrak{s}$. It returns an RLWE ciphertext encrypting $\mathfrak{m}_1 \cdot \mathfrak{m}_2 \in \mathcal{R}_p$. The standard deviation of the error increases to $\sigma_\times$.

We also introduce single-server PIR, which is used in the document retrieval step (§2.1).

PrivateInformationRetrieval [21]

Input: A set of documents $\mathcal{D} := (\mathsf{doc}_i)$ with $i \in [D]$, requested document index i'.
Output: The document $\mathsf{doc}_{i'}$.
Description: PIR is a protocol between a client and a server, where the server holds $\mathcal{D}$ and the client holds i'. The client retrieves $\mathsf{doc}_{i'}$ without revealing i' to the server. The server learns nothing about which document is retrieved.

3 Mímir's Construction

3.1 Overview

Setup. The protocol fixes a pretrained machine learning model that maps text strings to semantic embedding vectors: strings that are close in meaning produce vectors that are close in the embedding space (see §2.1).

The server runs its documents through the model to obtain their semantic embeddings. To store the embeddings, the server groups the vectors into clusters, based on the vectors' proximities to each other. For each cluster, the server encodes the vector embeddings as a collection of polynomials. The number of polynomials matches the embedding dimension. Each polynomial is formed by aggregating the corresponding coordinate across all vectors in the cluster and using these values as its coefficients. Figure 2 shows how the server's embedding database is configured.

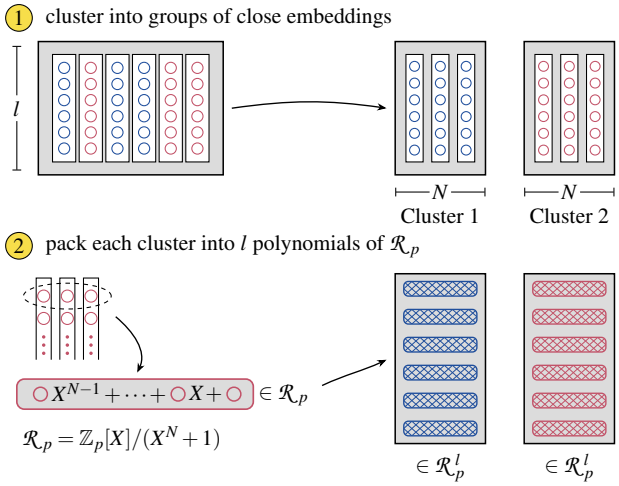


Figure 2: Overview of Mímir's database structure.

The server computes the centroid of each cluster and publishes the list of cluster centroids.

Finding the relevant cluster. A client who wants to do private search runs its query string through the machine learning model to obtain a query embedding. Then, the client selects the cluster whose centroid is closest to its query embedding.

Preparing the query. At this point, the client needs to encrypt two pieces of information and send them to the server. The first is the query embedding, so that the server can compute its similarity scores with the document embeddings under encryption. The second is the index of the cluster relevant to the client's search, so that the client can receive the scores for the documents from the relevant cluster. In cleartext, these leak information about what the client is searching for.

For the former, the client generates an LWE encryption of the query embedding. For the latter, the client generates an LWE′ encryption of a unit vector marking the index of the relevant cluster. Together, they form the client's query.

Computing document scores under encryption. Figure 3 illustrates the scoring retrieval steps for a query. Having received the encrypted query embedding, the server switches each LWE sample to an RLWE ciphertext using LWEtoRLWE (see §2.3), such that each RLWE ciphertext encrypts an entry of the query embedding.

The server now plaintext-multiplies each RLWE ciphertext with the embedding polynomials in each cluster, and then sums up the resulting RLWE ciphertexts. This realizes a component-wise multiplication between the query embedding and the document embeddings followed by summation, thereby computing inner products under encryption. Execut-

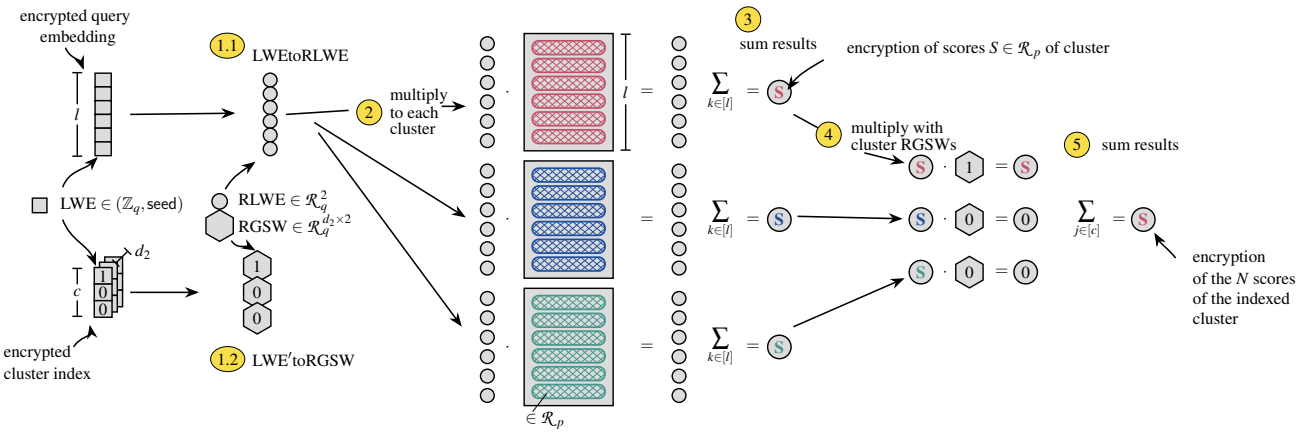


Figure 3: Scoring retrieval overview.

ing this procedure over all clusters produces encrypted document scores for all documents in all clusters.

Returning relevant document scores. To return only the scores of the relevant cluster, the server starts by converting each component of the LWE′-encrypted unit vector into an RGSW ciphertext. Only one of the RGSW ciphertexts encrypts a one, and the rest encrypt a zero. The server multiplies each RGSW ciphertext with the RLWE ciphertext containing the corresponding cluster's scores. Aggregating the results, the server obtains one RLWE ciphertext that contains the document scores only for the relevant cluster. The server returns this RLWE ciphertext to the client.

Retrieving relevant documents. Given the similarity scores, the client assumes the documents with the highest scores are most relevant for its search. The client uses an off-the-shelf PIR protocol to retrieve them, e.g., HintlessPIR [45].

3.2 Optimizations

3.2.1 Minimizing Communication

We minimize communication using two techniques: discarding irrelevant cluster scores before they reach the client [36] and transmitting compact ciphertexts.

Seeded ciphertexts. The random part of an LWE ciphertext is uniformly random and does not have to be transmitted as is. The client instead sends a short seed, which the server expands via a pseudorandom generator to reconstruct the random part. Multiple random parts belonging to different ciphertexts are reconstructed from a single seed.

LWE-to-RLWE switching. We achieve a further reduction in communication cost by letting the client upload LWE ciphertexts instead of RLWE ciphertexts. LWE ciphertexts are more compact than RLWE ciphertexts when the encrypted message consists of only one symbol. Thus, we let the client upload an LWE encryption of the query and an LWE′ encryption of the cluster index. The server then converts these to RLWE and RGSW ciphertexts respectively, before computing the document scores under encryption.

Cluster selection via RLWE×RGSW. To further minimize communication, the server computes over RLWE ciphertexts. Unlike LWE, RLWE ciphertexts support homomorphic multiplications with RGSW ciphertexts via RLWE×RGSW. This way, when aggregating the document scores over clusters, the server only keeps the scores for the relevant cluster, discarding the scores from other clusters by multiplying them with zero. Without RGSW, the client would have to upload individual encryptions of a zero vector for all but one cluster. This way, the client transmits $l + d_2 c$ LWE samples instead of lc, a technique used in [52, 58].

3.2.2 Computing in the NTT Domain

To minimize latency, all polynomial computations happen in the NTT domain. Before querying starts, the client generates its scheme-switching keys and the server stores the document embeddings in the NTT domain. At query time, the server only needs to transform the comparatively small query and cluster-selection ciphertexts; the document embeddings require no further transformation. Thus, scoring computation runs in time linear with the database size.

3.2.3 Plaintext Packing via CRT

Using CRT, we decompose the plaintext space into component rings with much smaller moduli. This improves the DFR significantly, allowing us to either use smaller parameters, or larger embedding vectors for Mímir. Smaller parameters lead to smaller communication whereas larger vectors generally lead to higher search quality.

More concretely, we pick r pairwise coprime moduli $p_0, \ldots, p_{r-1}$ such that $\prod_{i=0}^{r-1} p_i \geq p$, and represent a message $m \in \mathbb{Z}_p$ by its residues $m_i := m \bmod p_i \in \mathbb{Z}_{p_i}$, $i \in [r]$. By CRT, the tuple $(m_0, \ldots, m_{r-1})$ uniquely determines m, since

$m < p \leq \prod_i p_i$. Instead of a single ciphertext encrypting m under plaintext modulus p, we use r ciphertexts, one per component, where the ith ciphertext encrypts the residue m_i under the smaller plaintext modulus p_i. Every ciphertext and every operation of Algorithm 6 is instantiated once per component modulus, with all components computed independently; the client recovers m from the decrypted residues $(m_0, \ldots, m_{r-1})$ by CRT reconstruction. As each component ring corresponds to uploading and downloading an additional ciphertext, we keep the number of component rings small. Using two component rings is enough to observe major benefits. We apply CRT only to those ciphertexts whose plaintext space is the full $\mathbb{Z}_p$, such as the query embedding encryption. We do not apply it to the cluster selection ciphertexts, since these encrypt either 0 or 1. Splitting their plaintext space would add redundant ciphertexts and communication for no DFR benefit.

3.2.4 Supporting Arbitrary Cluster Sizes

Multiplying an RLWE ciphertext with a cluster polynomial requires both to lie in $\mathcal{R}_q$. This restricts the cluster polynomials to degree $< N$, capping each cluster at N documents, forcing at least $c = \lceil D/N \rceil$ clusters. But a large number of clusters means a small cluster size, decreasing the number of scores received, harming the search quality.

Mímir decouples N and the cluster size by setting the number of clusters independently and splitting each cluster into $\lceil D/(cN) \rceil$ *splits* of at most N documents. Each split is then encoded as a vector of polynomials over $\mathcal{R}_p^l$.

3.3 Mímir's Scoring Retrieval

The scoring retrieval protocol is presented in Algorithm 2. This algorithm identifies which documents in the server's database are most relevant for the client's search. We treat the LWE- and RLWE-parameter tuples (n, q, σ) and (N, q, σ) as ambient variables achieving λ bits of security.

Algorithm 2: $\mathsf{ScoringRetrieval}(B_1, B_2, \mathcal{D}, \mathsf{M}, l, \rho, \mathsf{query})$

Input: decomposition bases B_1 and B_2, a pretrained model M, embedding length l and precision ρ, a set of documents $\mathcal{D} = (\mathsf{doc}_i)_{i \in [D]}$, a query
Output: similarity scores $\mathbf{r}$

$\mathbf{s}, \mathfrak{s}, \mathbf{K}_1, \mathbf{K}_2 \leftarrow \mathsf{ClientSetup}(B_1, B_2)$;
$\mathbf{L}, \mathbf{P} \leftarrow \mathsf{ServerSetup}(\mathcal{D}, \mathsf{M}, l, \rho)$;
$\mathsf{ct}_\mathbf{q}, \mathsf{ct}_\mathbf{u} \leftarrow \mathsf{Query}(\mathsf{M}, l, \rho, \mathbf{s}, B_2, \mathbf{L}, \mathsf{query})$;
$\mathsf{ct}_{\mathsf{scores}} \leftarrow \mathsf{Response}(\mathbf{K}_1, \mathbf{K}_2, \mathbf{P}, \mathsf{ct}_\mathbf{q}, \mathsf{ct}_\mathbf{u})$;
$\mathbf{r} \leftarrow \mathsf{Recovery}(\mathsf{ct}_{\mathsf{scores}}, \mathfrak{s})$;
return $\mathbf{r}$

We describe below the algorithms in ScoringRetrieval. For ease of presentation, we omit (i) seeded ciphertexts (§3.2.1), (ii) computing in the NTT domain (§3.2.2), (iii) CRT packing (§3.2.3) – we present the algorithm as if it operates directly over the full plaintext space, whereas each operation is in practice performed independently over the CRT component rings, and (iv) splits (§3.2.4) – we assume $c = D/N$ clusters, each containing N documents.

To initiate the protocol, the client runs ClientSetup (Algorithm 3). The algorithm generates an LWE secret key $\mathbf{s}$ and its corresponding RLWE secret key $\mathfrak{s}$ such that $\mathsf{Coefs}(\mathfrak{s}) = \mathbf{s}$. This implies $n = N$ throughout the protocol (see §2.3), yet we treat them separately for clarity. The algorithm also generates the corresponding RLWE-switch and RGSW-switch keys $\mathbf{K}_1$ and $\mathbf{K}_2$. The secret key $\mathbf{s}$ and thus $\mathbf{K}_1$ and $\mathbf{K}_2$ can be reused: the client has to execute ClientSetup and transmit $\mathbf{K}_1$ and $\mathbf{K}_2$ only once to make an arbitrary number of queries.

Algorithm 3: $\mathsf{ClientSetup}(B_1, B_2)$

Input: decomposition bases B_1 and B_2
Output: LWE secret key $\mathbf{s}$, RLWE secret key $\mathfrak{s}$ with $\mathsf{Coefs}(\mathfrak{s}) = \mathbf{s}$, RLWE-switch key $\mathbf{K}_1$, RGSW-switch key $\mathbf{K}_2$

$\mathbf{s} \xleftarrow{R} \{-1, 0, 1\}^n$;
$\mathfrak{s} \leftarrow \sum_{i=0}^{N-1} \mathbf{s}[i] X^i$;
$\mathbf{K}_1 \leftarrow \mathsf{LWEtoRLWE-KeyGen}(\mathbf{s}, B_1)$;
$\mathbf{K}_2 \leftarrow \mathsf{RLWEtoRGSW-KeyGen}(\mathfrak{s}, B_2)$;
return $\mathbf{s}$, $\mathfrak{s}$, $\mathbf{K}_1$, $\mathbf{K}_2$

The ClientSetup protocol is followed by ServerSetup (Algorithm 4). For all of its documents $\mathsf{doc}_i \in \mathcal{D}$, $i \in [D]$, the server generates semantic embeddings $\mathbf{v}_i$ using the model M. These embeddings are then grouped into clusters $\mathbf{C}_j$, $j \in [c]$ of fixed size using a simple k-means clustering algorithm $\mathsf{Cluster}$ [39, 53]. For each cluster, a centroid $\mathbf{c}_j$ is computed by averaging over all the embeddings in the cluster. The server publishes the list of centroids $\mathbf{L}$.

To store its document embeddings, for each cluster, the server builds polynomials $\mathfrak{p}_k$ for all $k \in [l]$ out of the kth entries of the cluster embeddings. The server thus stores a vector of l polynomials $\mathbf{p}_j$, $j \in [c]$ for each cluster. ServerSetup has to be executed only once; adding, removing, or updating a document does not require a re-run. The server computes the new document's embedding, transforms it into the NTT domain, and writes it into the closest cluster's polynomials (or starts a new split if the affected split is full, §3.2.4).

With the setup completed, the client can now do its search by running Query (Algorithm 5). This algorithm starts by using M to generate an embedding $\mathbf{q}$ of the query. Then, the search-relevant cluster j' is determined as the cluster whose centroid is closest to $\mathbf{q}$ in the embedding space. The client encodes this information in a unit vector $\mathbf{u}_{j'}$. To wrap up, the client generates

$$\mathsf{ct}_\mathbf{q} = \mathrm{LWE}(\mathbf{s}, \mathbf{q}) \quad \text{and} \quad \mathsf{ct}_\mathbf{u} = \mathrm{LWE}'(\mathbf{s}, \mathbf{u}_{j'}),$$

Algorithm 4: $\mathsf{ServerSetup}(\mathcal{D}, \mathsf{M}, l, \rho)$

Input: a pretrained model M, embedding length l and precision ρ, a set of documents $\mathcal{D} = (\mathsf{doc}_i)_{i\in[D]}$,
Output: list of cluster centroids $\mathbf{L} = (\mathbf{c}_j)_{j\in[c]}$, per-cluster embedding polynomials $\mathbf{P} = (\mathbf{p}_j)_{j\in[c]}$

for $i \leftarrow 0$ **to** $D-1$ **do**
  $\mathbf{v}_i \leftarrow \mathsf{GenerateEmbedding}(\mathsf{M}, l, \rho, \mathsf{doc}_i)$;
$(\mathbf{C}_0, \ldots, \mathbf{C}_{c-1}) \leftarrow \mathsf{Cluster}((\mathbf{v}_0, \ldots, \mathbf{v}_{D-1}))$;
for $j \leftarrow 0$ **to** $c-1$ **do**
  $(\mathbf{v}_0, \ldots, \mathbf{v}_{N-1}) \leftarrow \mathbf{C}_j$;
  $\mathbf{c}_j \leftarrow \mathsf{sum}((\mathbf{v}_i)_{i\in[N]})/N$;
  for $k \leftarrow 0$ **to** $l-1$ **do**
    $\mathfrak{p}_k \leftarrow \sum_{i=0}^{N-1} \mathbf{v}_i[k] \cdot X^i$;
  $\mathbf{p}_j \leftarrow (\mathfrak{p}_0, \ldots, \mathfrak{p}_{l-1})$;
return $\mathbf{L} := (\mathbf{c}_0, \ldots, \mathbf{c}_{c-1}), \mathbf{P} := (\mathbf{p}_0, \ldots, \mathbf{p}_{c-1})$

which are the LWE encryption of the query embedding and the LWE′ encryption of the cluster selection.

Algorithm 5: $\mathsf{Query}(\mathsf{M}, l, \rho, \mathbf{s}, B_2, \mathbf{L}, \mathsf{query})$

Input: a pretrained model M, embedding length l and precision ρ, LWE secret key $\mathbf{s}$, LWE′ decomposition base B_2, list of cluster centroids $\mathbf{L} = (\mathbf{c}_j)_{j\in[c]}$, a query
Output: ciphertext $\mathsf{ct}_\mathbf{q}$ encrypting the query embedding, ciphertext $\mathsf{ct}_\mathbf{u}$ encrypting the cluster selection

$\mathbf{q} \leftarrow \mathsf{GenerateEmbedding}(\mathsf{M}, l, \rho, \mathsf{query})$;
$j' \leftarrow \mathrm{argmin}_{j\in[c]} \|\mathbf{q} - \mathbf{c}_j\|$;
return $\mathsf{ct}_\mathbf{q} := \mathrm{LWE}(\mathbf{s}, \mathbf{q})$, $\mathsf{ct}_\mathbf{u} := \mathrm{LWE}'(\mathbf{s}, \mathbf{u}_{j'})$

Upon receiving a query, the server runs Response (Algorithm 6). This algorithm starts by converting each component of $\mathsf{ct}_\mathbf{q}$ into an RLWE ciphertext

$$\mathsf{ct}_{\mathbf{q}[k]} = \mathrm{RLWE}_{\sigma_R}(\mathfrak{s}, \mathbf{q}[k]), \quad \forall k \in [l]$$

and each component of $\mathsf{ct}_\mathbf{u}$ into an RGSW ciphertext

$$\mathsf{ct}_{\mathbf{u}[j]} = \mathrm{RGSW}_{\sigma_G}(\mathfrak{s}, \mathbf{u}[j]), \quad \forall j \in [c].$$

where $\mathbf{q}[k], \mathbf{u}[j] \in \mathcal{R}_p$ are treated as constant-term polynomials. Then, for all clusters $j \in [c]$ and for all $k \in [l]$, the kth cluster polynomial $\mathfrak{p}_k$ plaintext-multiplies the kth RLWE ciphertext $\mathsf{ct}_{\mathbf{q}[k]}$. This corresponds to an element-wise multiplication of the query embedding with the corresponding entries of all the embedding vectors in a cluster. Summing up the resulting RLWE ciphertexts emulates an inner product under encryption, resulting in $\mathsf{ct}_{\mathbf{r}_j}$: the encryption of a polynomial, where the coefficients hold the similarity score for each document in cluster j. Multiplying each $\mathsf{ct}_{\mathbf{r}_j}$ with its corresponding RGSW ciphertext $\mathsf{ct}_{\mathbf{u}[j]}$ and summing the results gives a single RLWE ciphertext $\mathsf{ct}_{\mathsf{scores}}$ encrypting the similarity scores for cluster j'. Recall that for all $j \neq j'$, $\mathsf{ct}_{\mathbf{u}[j]}$ encrypts a zero, and $\mathsf{ct}_{\mathbf{u}[j']}$ encrypts a one.

Algorithm 6: $\mathsf{Response}(\mathbf{K}_1, \mathbf{K}_2, \mathbf{P}, \mathsf{ct}_\mathbf{q}, \mathsf{ct}_\mathbf{u})$

Input: RLWE-switch key $\mathbf{K}_1$, RGSW-switch key $\mathbf{K}_2$, list of document embeddings $\mathbf{P} = (\mathbf{p}_j)_{j\in[c]}$, LWE ciphertext $\mathsf{ct}_\mathbf{q}$, LWE′ ciphertext $\mathsf{ct}_\mathbf{u}$
Output: RLWE ciphertext $\mathsf{ct}_{\mathsf{scores}}$ encrypting the scores for the relevant cluster

for $k \leftarrow 0$ **to** $l-1$ **do**
  $\mathsf{ct}_{\mathbf{q}[k]} \leftarrow \mathsf{LWEtoRLWE}(\mathsf{ct}_\mathbf{q}[k], \mathbf{K}_1)$;
for $j \leftarrow 0$ **to** $c-1$ **do**
  $\mathsf{ct}_{\mathbf{u}[j]} \leftarrow \mathsf{LWE'toRGSW}(\mathsf{ct}_\mathbf{u}[j], \mathbf{K}_1, \mathbf{K}_2)$;
  $(\mathfrak{p}_0, \ldots, \mathfrak{p}_{l-1}) \leftarrow \mathbf{p}_j$;
  $\mathsf{ct}_{\mathbf{r}_j} \leftarrow \mathsf{sum}((\mathfrak{p}_k \cdot \mathsf{ct}_{\mathbf{q}[k]})_{k\in[l]})$;
$\mathsf{ct}_{\mathsf{scores}} \leftarrow \mathsf{sum}((\mathsf{RLWE} \times \mathsf{RGSW}(\mathsf{ct}_{\mathbf{r}_j}, \mathsf{ct}_{\mathbf{u}[j]}))_{j\in[c]})$;
return $\mathsf{ct}_{\mathsf{scores}}$

Finally, the client runs Recovery (Algorithm 7) on the server's response. This algorithm decrypts $\mathsf{ct}_{\mathsf{scores}}$ to reveal the similarity scores for cluster j'.

Algorithm 7: $\mathsf{Recovery}(\mathsf{ct}_{\mathsf{scores}}, \mathfrak{s})$

Input: RLWE ciphertext $\mathsf{ct}_{\mathsf{scores}} = (\mathfrak{a}, \mathfrak{b})$, RLWE secret key $\mathfrak{s}$
Output: similarity scores for the relevant cluster $\mathbf{r}$

$\mathbf{r} \leftarrow \lfloor (\mathfrak{b} + \mathfrak{a} \cdot \mathfrak{s})/\Delta \rceil$;
return $\mathbf{r}$

The helper algorithm GenerateEmbedding (Algorithm 8) maps any string into the plaintext space of Mímir. It starts by inputting the string into M to generate a semantic embedding. The semantic embedding is reduced to length l by applying principal component analysis (PCA). Then the algorithm quantizes all embedding entries to integers in the range $[-\psi, \psi-1]$ for $\psi := 2^{\rho-1}$.

Algorithm 8: $\mathsf{GenerateEmbedding}(\mathsf{M}, l, \rho, \mathsf{string})$

Input: a pretrained model M, embedding length l and precision ρ, a string
Output: embedding vector $\mathbf{v} \in \mathbb{Z}_p^l$

$\mathbf{v}'' \leftarrow \mathsf{M}(\mathsf{string})$;
$\mathbf{v}' \leftarrow \mathsf{PCA}(l, \mathbf{v}'')$;
$\mathbf{v} \leftarrow \mathsf{quantize}(\psi, \mathbf{v}')$;
return $\mathbf{v}$

3.4 Security

Mímir is a private search protocol that guarantees *search query privacy* against a computationally bounded *malicious*[1] adversary $\mathcal{A}$. Mímir achieves this by ensuring that the queries leave the client only in encrypted form. Hence, the security of the protocol is entirely based on the security of the underlying encryption schemes.

Formally, we define the notion of search query privacy similar to the security notions of *query privacy* [36] and *oblivious document ranking and retrieval* [2]. During a protocol run on an input query string m, the adversary $\mathcal{A}$ has access to the set of all public protocol parameters and protocol states $\mathcal{S}_{\mathsf{m}} = \{$public and evaluation keys, received and sent messages, intermediate server computation results$\}$.

Definition 2 (Search Query Privacy)**.** Let λ be a security parameter, let $\mathsf{m}_0, \mathsf{m}_1 \in \mathcal{M} = \{0,1\}^*$ be two query strings, and let $\mathcal{S}_{\mathsf{m}_i}$, $i \in [2]$, be the corresponding protocol states. A search protocol satisfies search query privacy if, for a uniformly random $b \in [2]$, every adversary $\mathcal{A}$ given $\mathcal{S}_{\mathsf{m}_b}$ running in time at most 2^{λ} outputs b with advantage

$$\left| \Pr\left[\mathcal{A}\left(\mathcal{S}_{\mathcal{A},\mathsf{m}_b}\right) = b\right] - \frac{1}{2} \right| \leq 2^{-\lambda}.$$

The first step of Mímir, scoring retrieval, achieves the indistinguishability of two queries by encrypting the corresponding embedding vectors $\mathbf{q}_i = \mathsf{GenerateEmbedding}(\mathsf{M}, l, \rho, \mathsf{string}_i)$ and the cluster selection vectors $\mathbf{u}_i$ with an IND-CPA secure homomorphic encryption scheme. By definition, the encryptions $\mathsf{ct}_{\mathbf{q}_i}, \mathsf{ct}_{\mathbf{u}_i}$ are semantically secure, and since all intermediate values are computed by the server as a deterministic function of these ciphertexts and public parameters, they remain computationally indistinguishable. The second step of the protocol, document retrieval, retains indistinguishability of two queries by employing a single-server PIR protocol satisfying query privacy [43]. By a standard hybrid argument over the two rounds, any distinguisher for Mímir yields a distinguisher for either the LHE scheme or the PIR protocol.

Mímir's scoring retrieval is instantiated with the Regev-style LHE scheme [48, 64] utilizing ciphertext switching schemes [16, 26]. These assume IND-CPA security and circular security [13], a specific type of key-dependent message (KDM) security [10], based on the hardness of GLWE. For detailed security reductions, we refer the reader to the corresponding papers. For its document retrieval, Mímir can use any single-server PIR protocol. A PIR that relies on GLWE minimizes the number of Mímir's underlying hardness assumptions.

[1] In contrast to privacy, Mímir provides correctness only against an honest-but-curious server.

3.5 Correctness

Mímir's scoring retrieval is *correct* if the client exactly recovers the similarity scores for the relevant cluster. This depends on two conditions. First, Response must select the right cluster and compute the scores correctly, which we state below in Proposition 1. Second, the error accumulated in the result ciphertext $\mathsf{ct}_{\mathsf{scores}}$ must remain below the decryption threshold of $\Delta/2$ (see §2.2), which we establish in Theorem 1. We defer all proofs to §A.

Proposition 1. *Let $\mathbf{q} \in \{-\psi, \ldots, \psi-1\}^l$ be the query embedding of the client and let $(\mathbf{v}_i)_{i\in[N]} \in \{-\psi, \ldots, \psi-1\}^l$ be the embedding vectors in cluster j. If the plaintext modulus $p > 2l\psi^2$ and $\mathsf{ct}_{\mathsf{scores}}$ decrypts correctly, then* Response *(Algorithm 6) outputs an RLWE ciphertext encrypting the polynomial whose i^{th} coefficient is $\langle \mathbf{q}, \mathbf{v}_i \rangle$.*

Noise growth. The two ciphertexts that make up the query, $\mathsf{ct}_{\mathbf{q}}$ and $\mathsf{ct}_{\mathbf{u}}$, are fresh LWE encryptions produced by the client. All fresh LWE samples contain an error term $e \xleftarrow{\chi_e} \mathbb{Z}_q$, where $\mathcal{X}_e$ is a discrete Gaussian distribution with variance σ^2 (see §2.2). During Response, this error grows due to homomorphic operations: LWEtoRLWE is applied to each $\mathsf{ct}_{\mathbf{q}}[k]$ and LWE′toRGSW is applied to each $\mathsf{ct}_{\mathbf{u}}[j]$. This is followed by the plaintext multiplications by the cluster polynomials $\mathfrak{p}_k$, the sum over l embedding entries, and the RLWE×RGSW multiplications followed by the sum over c clusters.

We list these operations and their effects on the input variance in Table 1 (summation simply adds the variances of the summands). Combining the operations, we obtain the bound in Theorem 1, which we use to bound the DFR of Mímir's scoring retrieval.

Table 1: Effects of operations in Response on the error variance. The variance of a fresh LWE ciphertext is denoted by σ^2. The variances of the input RLWE and RGSW ciphertexts to an algorithm are denoted by σ_{R}^2 and σ_{G}^2, respectively. The symbol $\psi = 2^{\rho-1}$ denotes the bound on the magnitude of the cluster polynomial coefficients.

Operation	Output Variance
LWEtoRLWE	$n^3 B_1^2 d_1 \sigma^2/12$
LWE′toRGSW	$n(n^3 B_1^2 d_1/3 + B_2^2 d_2)\sigma^2/4$
$\mathfrak{p}_k \cdot \mathsf{ct}_{\mathbf{q}[k]}$	$\psi^2 N \sigma_{\mathsf{R}}^2$
RLWE×RGSW	$N d_2 B_2^2 \sigma_{\mathsf{G}}^2/2 + \sigma_{\mathsf{R}}^2$

Theorem 1. *Let $\mathsf{ct}_{\mathbf{q}}$ and $\mathsf{ct}_{\mathbf{u}}$ be fresh LWE and LWE′ ciphertexts with dimension n, modulus q and error variance σ^2, respectively. Let $\mathsf{ct}_{\mathbf{q}}$ and $\mathsf{ct}_{\mathbf{u}}$ be the two ciphertext inputs to* Response *(Algorithm 6). The error variance of the output ciphertext $\mathsf{ct}_{\mathsf{scores}}$ is then bounded as*

$$\sigma_{\mathsf{scores}}^2 < cn^2 \left(n^3 B_1^2 B_2^2 d_1 d_2 + B_2^4 d_2^2 + n^2 l \psi^2 B_1^2 d_1\right) \sigma^2,$$

where c is the number of clusters, B_1 and B_2 are decomposition bases, $d_1 = \lfloor \log_{B_1} q \rfloor + 1$, $d_2 = \lfloor \log_{B_2} q \rfloor + 1$, and ψ is an upper bound on the absolute value of the plaintext polynomials' coefficients.

Remark 1. The bound above is worst case under every deterministic quantity: we assume maximal secret weight, full-weight gadget decompositions and maximum database magnitude. Unlike in [16], we also assume no distribution on the gadget decomposition digits. This only increases the variance of the error terms, whose Gaussian shape is preserved by the linear operations in Response. Thus, our DFR bound holds independently of any adversarial database content choice. We treat the error coefficients under the standard heuristic that they are independent [16,26]. This lets us add variances across summations.

Remark 2. Under CRT packing (§3.2.3), Theorem 1 stays unaffected, but the plaintext modulus p in Corollary 1 is replaced by the largest of the smaller component ring moduli.

Corollary 1. *Let $\Delta = \lfloor q/p \rceil$ and $\mathsf{ct}_{\mathsf{scores}}$ be bounded as in Theorem 1. Then, $\mathsf{ct}_{\mathsf{scores}}$ decrypts correctly except with probability at most*

$$N \cdot \operatorname{erfc}\left(\frac{\Delta}{2\sqrt{2}\sigma_{\mathsf{scores}}}\right).$$

Taking the optimizations in Sections 3.2.3 and 3.2.4 into account, for r CRT component rings and $\lceil D/(Nc) \rceil$ splits, the quantity above scales up by $r\lceil D/(Nc) \rceil$.

4 Evaluation

4.1 Implementation

We implement the database preprocessing in Python, as it has no effect on latency. For the embedding generation, any Hugging Face transformer model [76] can be specified. Clustering is performed via `faiss` k-means [28,40]. To generate clusters of the same size, we recursively apply k-means to the clusters with more than $\lceil D/c \rceil$ elements, and reassign some of the subclusters to other clusters, until all clusters contain at most $\lceil D/c \rceil$ documents. We apply *principal component analysis* (PCA) over the set of all embeddings – projecting them into an l-dimensional space – and uniformly quantize the float32 embedding entries into signed ρ-bit integers (from $[-\psi, \psi-1]$) while clipping values below the 10th and above the 90th percentile.

The cryptographic operations in Mímir are implemented in C++, building on the AVX/HEXL-optimized FHE-Deck library [42]. We provide latency benchmarks for sequential (single-threaded) and multithreaded executions, and in a simulated setup with parallelization over multiple servers.

All source code to reproduce the following results is publicly available [1].

4.2 Test Setup

Performance metrics. We evaluate Mímir's performance in terms of search quality, communication cost, and latency. As Mímir's contribution is a new cryptographic scoring retrieval algorithm, we exclude the PIR step from all benchmarks and only evaluate Mímir's scoring retrieval. PACMANN returns only the indices of the most relevant documents, and Tiptoe benchmarks the document retrieval step separately. Tiptoe's evaluation shows that the scoring retrieval step is more costly than the document retrieval step in terms of both communication and latency [36].

To measure search quality, we use the *mean reciprocal rank* (MRR@100). For each query, Mímir returns a ranked list of 100 documents. The reciprocal rank of a query is the inverse of the position of the ground truth in this list, or 0 if it does not appear in it. We average this value over 1000 queries, yielding the MRR@100.

For communication, there are one-time and per-query costs. Per-query costs contribute to latency. The one-time cost is due to the two scheme-switching keys and excludes the protocol-independent embedding model (approximately 300 MB), and the publication of the configuration parameters for the PCA reduction and quantization.

We measure the latency as the time from encrypting a query embedding to retrieving the scores in plaintext. The protocol-independent embedding generation, which takes 5 ms on average, is excluded from all benchmarks. Latency benchmarks also exclude network transfer time, which adds to the latency depending on the client's bandwidth: at 100 Mbps, every 12 MiB of per-query communication adds about 1 s.

Evaluation set. We evaluate Mímir's performance over the MS MARCO [8,55] document ranking dataset. It consists of 3,213,835 documents of website content alongside 5,193 real Bing queries, where each query is paired with one human-annotated relevant document as *ground truth*. An evaluation over this dataset gives an idea of how Mímir would perform in web search.

Embedding model. We evaluate Mímir using the embedding model msmarco-distilbert-base-tas-b (distilbert) [76] for a fair comparison with the baselines. It maps text documents (truncated to a maximum sequence length of 512 tokens) to float32 embedding vectors of length 768. The model uses the inner product as its similarity metric.

Parameter selection. We search over all (n, q, σ) tuples that provide $\lambda \geq 128$ bits and DFR $\leq 2^{-40}$. We measure the security level of each tuple via the LWE Estimator [4]. We restrict $q < 2^{62}$ and choose it as a prime such that $q \equiv 1 \pmod{2n}$ for an optimal NTT. The concrete parameters used for our Mímir instantiations are listed in Table 3.

Hardware. We run all latency benchmarks on a single machine to eliminate network variance and ensure reproducibility. Our machine is equipped with an AMD Ryzen 9 7950X3D processor (16 cores / 32 threads, 4.2 GHz), 96 GB

RAM. The CPU supports AVX2/AVX-512, allowing the use of AVX/HEXL-optimized libraries for accelerated homomorphic operations.

Baselines. We compare Mímir to two other private search engines that operate under the same adversarial model.

Like Mímir, Tiptoe [36] relies on Regev-based (LWE) homomorphic encryption, and Mímir adopts its preprocessing pipeline (clustering, PCA reduction, quantization). This baseline allows us to isolate the performance differences stemming from Mímir's cryptographic construction. We omit Tiptoe's quality optimizations from both protocols, as they apply to either equally. For the same reason, we reproduce Tiptoe's search quality over MS MARCO with our own preprocessing, which yields a worse MRR@100 than the ≈0.14 reported in [36]. We compute Tiptoe's communication cost for MS MARCO ourselves, and reproduce its scoring retrieval latency on our hardware, running all processes on our machine.

Orthogonally, PACMANN [81] performs approximate nearest neighbor search – equivalent to the scoring computation – via graph traversal rather than clustering, and its privacy relies on one-way functions instead of homomorphic encryption. We reproduce its reported communication cost and latency for the MS MARCO dataset on our hardware, and the search quality is taken from [81, Table 1].

Offline and online costs. Prior work assumes the database does not change between queries. This way it can transfer part of the per-query computation and communication to a non-latency-critical path, reducing online latency significantly. We refer to costs incurred by these operations as offline costs. We assume a dynamic database, i.e., that it may change between any two queries. This means that these offline costs cannot be offloaded to a non-latency-critical path. Therefore, we report both the total and online latency, where the total latency is online + offline latency (see Table 2). Similarly, per-query communication is offline + online communication. Mímir has no offline phase, so its total and online latencies are the same, and all of its per-query communication is online.

4.3 Results

Mímir's search quality, communication cost, and latency are governed by three parameters: the number of clusters c, the embedding length l, and the embedding precision ρ.

The number of clusters c has a strong effect on quality: a larger c increases the probability of searching in the wrong cluster. Communication cost initially decreases with larger c as fewer scores are returned, until a breaking point where the upload cost keeps increasing due to c LWE′ ciphertexts but the download cost stays the same due to the minimum cluster size N (see Figure 4). Latency is lowest at an intermediate c: the LWE′-to-RGSW switches grow with c, while the response the server packs and the client decrypts shrinks with it. PCA reduction to embedding length l causes the second-largest quality loss. But a smaller l reduces both communication cost and latency due to fewer LWE-to-RLWE switches.

Embedding precision ρ can be reduced substantially before quality degrades. As a small ρ allows smaller LWE parameters, it improves both communication and latency. Each parameter's effect on search quality is detailed in §B.

Mímir I–IV. We present four instantiations of Mímir using the distilbert embedding model for a direct comparison to PACMANN and Tiptoe. Mímir I shows the communication it needs to match Tiptoe's search quality. Since Mímir and Tiptoe share the same preprocessing pipeline (semantic embeddings, PCA, quantization, clustering), the same c results in the same search quality, so this comparison isolates the cryptography. Mímir II shows the search quality it achieves by choosing the smallest c while staying roughly within Tiptoe's per-query communication budget (see Figure 4). Mímir III shows the per-query communication it requires to match PACMANN's search quality. For this, Mímir III has to stick to $c = 1$, otherwise the quality drops below PACMANN's. This is already the maximum search quality Mímir reaches with $l = 192$ and $\rho = 5$. Mímir IV shows the quality and communication it achieves while staying within PACMANN's total latency budget, which is the faster of the two baselines. The evaluation results are in Table 2.

Against Tiptoe. For the same search quality, Mímir I requires only 0.24 MiB, which is 84× less than Tiptoe's per-query communication (20.26 MiB). Mímir I has 4.23 s of latency, which is 16% slower than Tiptoe's 3.65 s total latency, and 52.9× slower than Tiptoe's 0.08 s online latency.

Mímir II roughly matches Tiptoe's communication and achieves an MRR@100 of 0.226. This corresponds to 64% of the model's non-private search quality as opposed to Tiptoe's 36%. Mímir II is much faster: it requires only 0.87 s. This is 4.2× smaller than Tiptoe's total latency, yet it remains 10.9× larger than Tiptoe's online latency.

Mímir IV outperforms Tiptoe on all performance metrics. It achieves an MRR@100 of 0.206, 59% of the model's non-private search quality. Its per-query communication of 8.13 MiB is 2.5× smaller than Tiptoe's, and its 0.77 s latency is 4.7× smaller than Tiptoe's total latency. In the online phase alone, it is 9.6× slower than Tiptoe.

Against PACMANN. Mímir III reaches an MRR@100 of 0.285, 81% of the non-private search quality, against PACMANN's 0.266 (76%). For this, Mímir III requires 44.46 MiB of per-query communication, which is 72% of PACMANN's (61.85 MiB). But Mímir III has higher latency: its 1.66 s is 2.1× slower than PACMANN.

Mímir IV is on par with PACMANN's total latency (0.77 s vs. 0.80 s), yet it is 12.8× slower than PACMANN in the online phase alone. Additionally, PACMANN's benchmark is single-threaded by choice, and it can be parallelized. But PACMANN needs around 20 round trips per query while Mímir needs one. Our latency benchmarks exclude round trips, but PACMANN's WAN evaluation simulates an additional 50 ms per round trip [81]. This amounts to an extra 0.95 s for 19 ad-

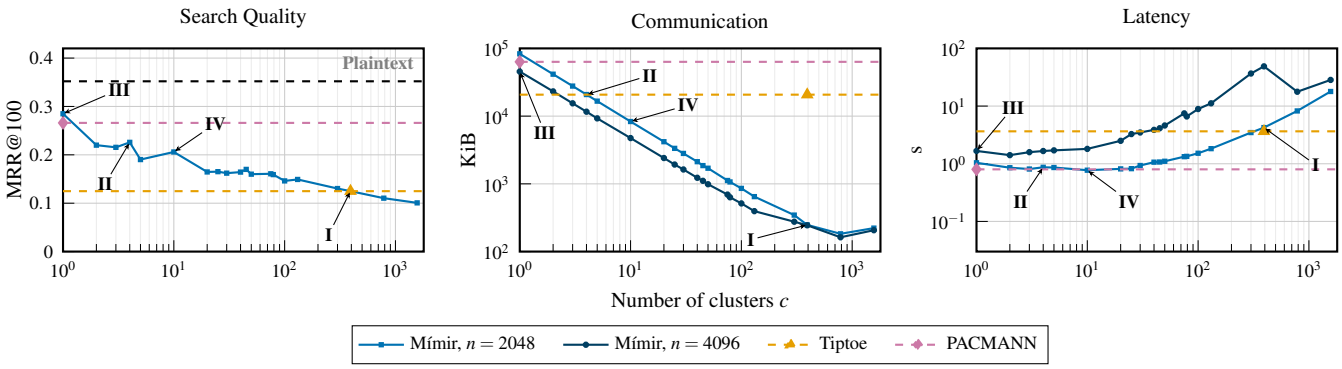


Figure 4: Search quality, per-query communication and latency across c for $l = 192$ and $\rho = 5$, for $n = 2048$ and $n = 4096$. Search quality does not depend on n. Latency is reported for the multithreaded execution. The baselines' communication and latency include their offline phase. Arrows mark Mímir I–IV. PACMANN does not cluster; we place it at $c = 1$.

Table 2: Search quality, communication and latency in the dynamic setting (see §4.2). All protocols use the distilbert model. In the dynamic setting, the baselines' offline phase lies on the latency-critical path.

Protocol	l	ρ	c	**MRR@100**	**Communication**		**Latency**		
					One-time	**Per-query**	**Offline**	**Online**	**Total**
Plaintext	768	float32	1	0.352	–	12.26 MiB	–	–	–
Tiptoe	192	5 bits	393	0.125	0.02 KiB	20.26 MiB	3.57 s$^\circ$	0.08 s$^\circ$	3.65 s$^\circ$
Mímir I	192	5 bits	393	0.125	5.01 MiB	0.24 MiB	–	4.23 s	4.23 s
Mímir II	192	5 bits	4	0.226	4.93 MiB	20.34 MiB	–	0.87 s	0.87 s
PACMANN	192	float32	1	0.266†	2.68 GiB*	61.85 MiB	0.74 s‡	0.06 s‡	0.80 s‡
Mímir III	192	5 bits	1	0.285	9.54 MiB	44.46 MiB	–	1.66 s	1.66 s
Mímir IV	192	5 bits	10	0.206	3.38 MiB	8.13 MiB	–	0.77 s	0.77 s

† PACMANN's MRR@100 is taken from Tab. 1 of [81]. ‡ In comparison to Tiptoe and Mímir, PACMANN's implementation is single-threaded. $^\circ$ Tiptoe's latency is benchmarked using the official implementation, reconfigured for optimal communication cost for MS MARCO. * PACMANN states in its limitations that it does not support dynamic database updates [81], implying that its one-time preprocessing must be redone after every update. In that case, in a dynamic setting, PACMANN's per-query communication becomes 2.68 GiB (the preprocessing plus 3.03 MiB of online communication), with no one-time communication. The 58.81 MiB of its 61.85 MiB per-query communication is the maintenance cost for the preprocessed state, which becomes redundant.

ditional round trips. Mímir IV reaches 59% of the non-private search quality against PACMANN's 76%, in exchange for 8.13 MiB of per-query communication, which is 7.6× smaller than PACMANN's.

One-time costs. Mímir's one-time upload of the client's scheme-switching keys is 3.38–9.54 MiB for Mímir I–IV, compared to Tiptoe's 0.02 KiB and PACMANN's 2.68 GiB.

Parallelization. Mímir's server-side computation is highly parallelizable. On top of multithreading, it can be distributed across a cluster of machines. Our simulations show that distributing the parallelizable workload across 10 machines reduces the latency of Mímir I from 4.23 s to 1.43 s, Mímir II from 0.87 s to 0.32 s and Mímir IV from 0.77 s to 0.25 s. For details on the single-threaded and distributed benchmarks, see Table 7 in the appendix.

Mímir-bge. Additionally, we instantiate Mímir using the bge-small-en-v1.5 (bge) [77] model. This model outputs normalized vector embeddings and also uses inner products to compute the similarity scores, i.e., it uses cosine similarity as its similarity metric (see §2). This shows how Mímir performs with a modern, more accurate embedding model, and that an improvement in the embedding model leads to a search with better quality. Mímir-bge uses $l = 256$, $\rho = 3$ bits and $c = 7$ for near-minimal latency. Mímir-bge reaches an MRR@100 of 0.304 with a one-time upload of 3.71 MiB, a per-query communication of 5.82 MiB, and a latency of 0.92 s. This is 80% of bge's non-private search quality (0.381).

5 Related Work

5.1 Private Search over Plaintext Data

We divide private search protocols into two categories: *semantic* search returns results with similar embedding vec-

tors [50,56,65], and *lexicographic* search is based on keyword matching [66] or tf-idf scores [67].

Semantic private search. Semantic private search protocols reduce the size of the candidate set for the scoring retrieval either via clustering [7, 15, 17, 36, 47] or graphs [81].

Tiptoe [36] enables private search under the same security guarantees as Mímir. It groups embedding vectors into clusters and privately retrieves inner-product similarity scores for the relevant cluster via linearly homomorphic encryption, using one-time offline tokens to reduce online communication.

Other work targets weaker security guarantees or different threat models. Wally [7] combines homomorphic encryption with differential privacy to reduce server load. PrivANN [17] targets private approximate nearest neighbor search (ANNS), the operation performed during scoring retrieval of private semantic search under trusted-execution-environment security. SANNS [15] and Panther [47] achieve private ANNS under purely cryptographic guarantees that also protect server privacy: the client learns nothing beyond the relevant results.

Graph-based semantic search [50] is the state of the art for unencrypted ANN search, outperforming clustering-based approaches. PACMANN [81] adapts this to the private setting by prioritizing search quality over communication efficiency: the client privately retrieves the graph structure via PIR in a one-time amortizable preprocessing step and traverses the graph locally. This yields higher search quality than clustering-based private search protocols.

Lexicographic private search. Coeus [2] is a private Wikipedia search engine based on tf-idf search, assuming the same adversarial model as Tiptoe and PACMANN. It reduces communication cost by extending the common two-round protocol to three rounds: an intermediate *keyword query* retrieves short descriptions of top results, followed by a classical PIR query for the single desired document. It uses a tailored homomorphic encryption scheme for the underlying tf-idf vector-matrix multiplication.

Multi-server private search. Private search [75] and private ANNS [69] have also been studied under multiple non-colluding servers.

5.2 Private Search over Encrypted Data

Unlike in §5.1, where clients query plaintext databases, Symmetric Searchable Encryption (SSE) [72] and Public-Key Encryption with Keyword Search (PEKS) [11] let a secret-key holder search their own encrypted data without downloading and decrypting it. These primitives have been extensively studied [23, 41, 73, 78], with applications including filesystem search [24], email search [79], and medical data storage [38].

Some of the SSE and PEKS protocols use semantic embeddings. HERS [29] enables private image recognition over encrypted images via inner-product and cosine similarity. Compass [82] is a secure ANNS protocol using a graph-based index (HNSW) rather than inner-product-over-clusters similarity, achieving state-of-the-art quality with ORAM-based privacy. Onyx [63] improves Compass under an additional trusted-execution-environment assumption.

A separate line of work performs lexicographic search over encrypted databases via secure pattern matching, optimized for server-side efficiency [12, 59].

5.3 Comparison to PIR and Keyword-PIR

Unlike private search, PIR [21, 43] retrieves an item by its known index, requiring the client to already know which item to fetch.[2] Despite early inefficiency [70], single-server PIR has since seen both asymptotic [31] and practical [3,5,6,25,37, 51, 52] improvements. Some schemes are *stateful*, requiring preprocessing to reduce online cost [22, 25, 37, 58, 60, 80] that has to be redone whenever the database is updated, while others are *stateless* [45].

Keyword-PIR [20] and keyword search [30] enable private retrieval by keyword (e.g., a hash of the document title). Unlike in PIR, where every queried index returns an entry, the keyword domain in Keyword-PIR is much larger than the database, so non-existent keywords return nothing. Unlike private search schemes, which return documents related to a query, these schemes [5, 14, 18, 35, 49, 61] require the client to know the exact keyword.

6 Conclusion

Mímir is a stateless private search engine: the client sends compact ciphertexts, and the server produces the similarity scores for the relevant cluster under encryption. Mímir incurs no database-dependent preprocessing, and the setup material is generated once and amortized over arbitrarily many queries.

Mímir can be instantiated across a range of trade-off points between search quality, communication cost, and latency. Compared to state-of-the-art private search protocols, Mímir achieves significantly lower communication cost or allows improvements in search quality, at the price of higher but parallelizable computation. Specifically, Mímir IV outperforms Tiptoe on all three performance metrics. Mímir provides the first semantic private search protocol for dynamic settings with limited bandwidth.

Limitations. To outperform PACMANN's search quality, Mímir has to forgo clustering, resulting in sub-optimal latency. Mímir can outperform PACMANN either in search quality and communication, or in communication and latency, but not all three simultaneously (see Mímir III and Mímir IV in Table 2). Mímir's computation remains linear in the database size, and in the online phase alone it is slower than protocols that shift work into a query-independent offline phase.

[2] Knowledge of item indices does not imply knowledge of the database contents; indices may come from public/compact mappings or hash functions.

References

[1] Gökberk Erdoğan, Emma Munisamy, and Kamil Kluczniak. Mimir implementation and parameter search scripts. https://github.com/Emmbert/mimir, 2026. GitHub repository.

[2] Ishtiyaque Ahmad, Laboni Sarker, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. Coeus: A system for oblivious document ranking and retrieval. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 672–690, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3477132.3483586.

[3] Ishtiyaque Ahmad, Yuntian Yang, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. Addra: Metadata-private voice communication over fully untrusted infrastructure. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 313–329. USENIX Association, July 2021. URL: https://www.usenix.org/conference/osdi21/presentation/ahmad.

[4] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology*, 9(3):169–203, October 2015. doi:10.1515/jmc-2015-0016.

[5] Asra Ali, Tancrède Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication–Computation trade-offs in PIR. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1811–1828. USENIX Association, August 2021. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/ali.

[6] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. PIR with Compressed Queries and Amortized Query Processing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 962–979, San Francisco, CA, May 2018. IEEE. URL: https://ieeexplore.ieee.org/document/8418648/, doi:10.1109/SP.2018.00062.

[7] Hilal Asi, Fabian Boemer, Nicholas Genise, Muhammad Haris Mughees, Tabitha Ogilvie, Rehan Rishi, Kunal Talwar, Karl Tarbe, Akshay Wadia, Ruiyu Zhu, and Marco Zuliani. Scalable private search with wally, 2025. URL: https://arxiv.org/abs/2406.06761, arXiv:2406.06761.

[8] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.

[9] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers computation in private information retrieval: PIR with preprocessing. In *Advances in Cryptology – CRYPTO 2000*, volume 1880 of *Lecture Notes in Computer Science*, pages 55–73. Springer, 2000. doi:10.1007/3-540-44598-6_4.

[10] John Black, Phillip Rogaway, and Thomas Shrimpton. Encryption-scheme security in the presence of key-dependent messages. In *International Workshop on Selected Areas in Cryptography*, pages 62–75. Springer, 2002.

[11] Dan Boneh, Giovanni Di Crescenzo, Rafail Ostrovsky, and Giuseppe Persiano. Public key encryption with keyword search. In Christian Cachin and Jan L. Camenisch, editors, *Advances in Cryptology - EUROCRYPT 2004*, pages 506–522, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[12] Charlotte Bonte and Ilia Iliashenko. Homomorphic string search with constant multiplicative depth. In *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop*, pages 105–117, 2020.

[13] J Camenisch and Anna Lysyanskaya. Efficient non-transferable anonymous multishow credential system with optional anonymity revocation. In *Advances in Cryptology-EUROCRYPT'01*, pages 93–118, 2001.

[14] Sofia Celi and Alex Davidson. Call me by my name: Simple, practical private information retrieval for keyword queries. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 4107–4121, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3670271.

[15] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadegh Riazi. {SANNS}: Scaling up secure approximate {k-Nearest} neighbors search. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2111–2128, 2020.

[16] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient Homomorphic Conversion Between (Ring) LWE Ciphertexts. In Kazue Sako and Nils Ole Tippenhauer, editors, *Applied Cryptography and Network Security*, volume 12726, pages 460–479. Springer International Publishing, Cham, 2021. Series Title: Lecture Notes in Computer Science. URL: https://link.springer.com/10.1007/978-3-030-78372-3_18, doi:10.1007/978-3-030-78372-3_18.

[17] Xuan Chen, Shujie Cui, Joseph K Liu, Shi-Feng Sun, and Shangqi Lai. Privann: Practical and efficient private

approximate nearest neighbor search. In *2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 1218–1226. IEEE, 2025.

[18] Zehao Chen, Zhaoyan Shen, Qian Wei, Hang Lu, and Lei Ju. Conflux: A high-performance keyword private retrieval system for dynamic datasets. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1–14. IEEE, 2026.

[19] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan. Private information retrieval. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*, pages 41–50, October 1995. ISSN: 0272-5428. doi:10.1109/SFCS.1995.492461.

[20] Benny Chor, Niv Gilboa, and Moni Naor. Private information retrieval by keywords. *IACR Cryptol. ePrint Arch.*, 1998:3, 1998.

[21] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *J. ACM*, 45(6):965–981, November 1998. doi:10.1145/293347.293350.

[22] Henry Corrigan-Gibbs, Alexandra Henzinger, and Dmitry Kogan. Single-server private information retrieval with sublinear amortized time. In *Advances in Cryptology – EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 – June 3, 2022, Proceedings, Part II*, page 3–33, Berlin, Heidelberg, 2022. Springer-Verlag. doi:10.1007/978-3-031-07085-3_1.

[23] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, CCS '06, page 79–88, New York, NY, USA, 2006. Association for Computing Machinery. doi:10.1145/1180405.1180417.

[24] Emma Dauterman, Eric Feng, Ellen Luo, Raluca Ada Popa, and Ion Stoica. DORY: An encrypted search system with distributed trust. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1101–1119. USENIX Association, November 2020. URL: https://www.usenix.org/conference/osdi20/presentation/dauterman-dory.

[25] Alex Davidson, Gonçalo Pestana, and Sofía Celi. FrodoPIR: Simple, Scalable, Single-Server Private Information Retrieval. *Proceedings on Privacy Enhancing Technologies*, 2023(1):365–383, January 2023. URL: https://petsymposium.org/popets/2023/popets-2023-0022.php, doi:10.56553/popets-2023-0022.

[26] Gabrielle De Micheli, Duhyeong Kim, Daniele Micciancio, and Adam Suhl. Faster Amortized FHEW Bootstrapping Using Ring Automorphisms. In Qiang Tang and Vanessa Teague, editors, *Public-Key Cryptography – PKC 2024*, volume 14604, pages 322–353. Springer Nature Switzerland, Cham, 2024. Series Title: Lecture Notes in Computer Science. URL: https://link.springer.com/10.1007/978-3-031-57728-4_11, doi:10.1007/978-3-031-57728-4_11.

[27] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. In *Proceedings of the 13th USENIX Security Symposium*, pages 303–320, 2004.

[28] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2024. arXiv:2401.08281.

[29] Joshua J Engelsma, Anil K Jain, and Vishnu Naresh Boddeti. Hers: Homomorphically encrypted representation search. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 4(3):349–360, 2022.

[30] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings*, volume 3378 of *Lecture Notes in Computer Science*, pages 303–324. Springer, 2005. URL: https://iacr.org/archive/tcc2005/3378_304/3378_304.pdf, doi:10.1007/978-3-540-30576-7_17.

[31] Craig Gentry and Shai Halevi. Compressible fhe with applications to pir. In Dennis Hofheinz and Alon Rosen, editors, *Theory of Cryptography*, pages 438–464, Cham, 2019. Springer International Publishing.

[32] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, volume 8042, pages 75–92, 2013. ISBN: 9783642400407 9783642400414. Place: Berlin, Heidelberg. Publisher: Springer Berlin Heidelberg. URL: http://link.springer.com/10.1007/978-3-642-40041-4_5, doi:10.1007/978-3-642-40041-4_5.

[33] Bryan Gleeson, Arthur Lin, Juha Heinanen, Grenville Armitage, and Andrew Malis. A framework for IP

based virtual private networks. RFC 2764, Internet Engineering Task Force, February 2000. URL: https://www.ietf.org/rfc/rfc2764.txt.

[34] Google. Understanding searches better than ever before, October 2019. Google Blog. URL: https://blog.google/products-and-platforms/products/search/search-language-understanding-bert/.

[35] Meng Hao, Weiran Liu, Liqiang Peng, Cong Zhang, Pengfei Wu, Lei Zhang, Hongwei Li, and Robert H. Deng. Practical keyword private information retrieval from key-to-index mappings. Cryptology ePrint Archive, Paper 2025/210, 2025. URL: https://eprint.iacr.org/2025/210.

[36] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private Web Search with Tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 396–416, Koblenz Germany, October 2023. ACM. URL: https://dl.acm.org/doi/10.1145/3600006.3613134, doi:10.1145/3600006.3613134.

[37] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval, 2022. Publication info: Published elsewhere. Major revision. USENIX Security Symposium 2023. URL: https://eprint.iacr.org/2022/949.

[38] Zhuobin Hu, Jiabei Wang, Zhengkai Chen, Zhaoxuan Ge, Mingyu Bian, Lei Chen, and Yongbin Zhou. Seac: dynamic searchable symmetric encryption with lightweight update-search permission control. *Cybersecurity*, 8(1):75, 2025. doi:10.1186/s42400-025-00370-9.

[39] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[40] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[41] Seny Kamara, Charalampos Papamanthou, and Tom Roeder. Dynamic searchable symmetric encryption. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, CCS '12, page 965–976, New York, NY, USA, 2012. Association for Computing Machinery. doi:10.1145/2382196.2382298.

[42] Kamil Kluczniak. FHE-Deck: A C++ library for fully homomorphic encryption based on functional bootstrapping. https://github.com/FHE-Deck/fhe-deck-core, 2023. GitHub repository.

[43] Eyal Kushilevitz and Rafail M. Ostrovsky. Replication is not needed: single database, computationally-private information retrieval. *Proceedings 38th Annual Symposium on Foundations of Computer Science*, pages 364–373, 1997.

[44] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[45] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz-Wu. Hintless single-server private information retrieval. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 183–217, Cham, 2024. Springer Nature Switzerland.

[46] Haoran Li, Mingshi Xu, and Yangqiu Song. Sentence embedding leaks more information than you expect: Generative embedding inversion attack to recover the whole sentence. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 14022–14040, 2023.

[47] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. Panther: Private approximate nearest neighbor search in the single server setting. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 365–379, 2025.

[48] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, pages 1–23, Berlin, Heidelberg, 2010. Springer. doi:10.1007/978-3-642-13190-5_1.

[49] Rasoul Akhavan Mahdavi and Florian Kerschbaum. Constant-weight PIR: Single-round keyword PIR via constant-weight equality operators. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1723–1740, Boston, MA, August 2022. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/mahdavi.

[50] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[51] Carlos Aguilar Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. Xpir: Private information retrieval for everyone. *Proceedings on Privacy Enhancing Technologies*, pages 155–174, 2016.

[52] Samir Jordan Menon and David J Wu. Spiral: Fast, high-rate single-server pir via fhe composition. In *2022 IEEE symposium on security and privacy (SP)*, pages 930–947. IEEE, 2022.

[53] Meta AI Research. Faiss building blocks: clustering, PCA, quantization. https://github.com/facebookresearch/faiss/wiki/Faiss-building-blocks:-clustering,-PCA,-quantization. Accessed: 2026-08-17.

[54] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. Cryptology ePrint Archive, Paper 2011/501, 2011. URL: https://eprint.iacr.org/2011/501.

[55] Microsoft. MS MARCO datasets for document and passage ranking leaderboards. https://microsoft.github.io/msmarco/Datasets. Accessed: 2026-08-05.

[56] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[57] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M Rush. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12448–12460, 2023.

[58] Muhammad Haris Mughees, Hao Chen, and Ling Ren. Onionpir: Response efficient single-server pir. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 2292–2306, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3460120.3485381.

[59] Shintaro Narisada, Hiroki Okada, Takashi Nishide, and Kazuhide Fukushima. Efficient homomorphic string search via TFHE. Cryptology ePrint Archive, Paper 2026/1016, 2026. URL: https://eprint.iacr.org/2026/1016.

[60] Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Private stateful information retrieval. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, page 1002–1019, New York, NY, USA, 2018. Association for Computing Machinery. doi:10.1145/3243734.3243821.

[61] Sarvar Patel, Joon Young Seo, and Kevin Yeo. Don't be dense: Efficient keyword PIR for sparse databases. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3853–3870, Anaheim, CA, August 2023. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/patel.

[62] Sai Teja Peddinti and Nitesh Saxena. On the privacy of web search based on query obfuscation: A case study of trackmenot. In *International Symposium on Privacy Enhancing Technologies Symposium*, pages 19–37. Springer, 2010.

[63] Deevashwer Rathee, Jean-Luc Watson, Zirui Neil Zhao, G Edward Suh, and Raluca Ada Popa. Onyx: Cost-efficient disk-oblivious ann search. *arXiv preprint arXiv:2604.20401*, 2026.

[64] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6):34:1–34:40, September 2009. URL: https://dl.acm.org/doi/10.1145/1568318.1568324, doi:10.1145/1568318.1568324.

[65] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[66] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[67] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[68] SAP. Embeddings & semantic search. SAP Architecture Center, January 2025. URL: https://architecture.learning.sap.com/docs/ref-arch/876f58.

[69] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. Private approximate nearest neighbor search with sublinear communication. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 911–929. IEEE, 2022.

[70] Radu Sion and Bogdan Carbunar. On the computational practicality of private information retrieval. In *Proceedings of the network and distributed systems security symposium*. Internet Society Geneva, Switzerland, 2007.

[71] Congzheng Song and Ananth Raghunathan. Information leakage in embedding models. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, pages 377–390, 2020.

[72] Dawn Xiaodong Song, David Wagner, and Adrian Perrig. Practical techniques for searches on encrypted data. In *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, SP '00, page 44, USA, 2000. IEEE Computer Society.

[73] Mikhail Strizhov and Indrajit Ray. Multi-keyword similarity search over encrypted cloud data. In Nora Cuppens-Boulahia, Frédéric Cuppens, Sushil Jajodia, Anas Abou El Kalam, and Thierry Sans, editors, *ICT Systems Security and Privacy Protection*, pages 52–65, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.

[74] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[75] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 299–313, Boston, MA, March 2017. USENIX Association. URL: https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/wang-frank.

[76] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. Huggingface's transformers: State-of-the-art natural language processing. *CoRR*, abs/1910.03771, 2019. URL: http://arxiv.org/abs/1910.03771, arXiv:1910.03771.

[77] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packaged resources to advance general chinese embedding, 2023. arXiv:2309.07597.

[78] Jiadi Yu, Peng Lu, Yanmin Zhu, Guangtao Xue, and Minglu Li. Toward secure multikeyword top-k retrieval over encrypted cloud data. *IEEE Trans. Dependable Secur. Comput.*, 10(4):239–250, July 2013. doi:10.1109/TDSC.2013.9.

[79] Wenzheng Zhang, Baodong Qin, Xinfeng Dong, and Aikui Tian. Public-key encryption with bidirectional keyword search and its application to encrypted emails. *Computer Standards & Interfaces*, 78:103542, 2021. URL: https://www.sciencedirect.com/science/article/pii/S0920548921000374, doi:10.1016/j.csi.2021.103542.

[80] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation . In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4296–4314, Los Alamitos, CA, USA, May 2024. IEEE Computer Society. URL: https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00055, doi:10.1109/SP54263.2024.00055.

[81] Mingxun Zhou, Elaine Shi, and Giulia Fanti. Pacmann: Efficient private approximate nearest neighbor search. Cryptology ePrint Archive, Paper 2024/1600, 2024. URL: https://eprint.iacr.org/2024/1600.

[82] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. Compass: Encrypted semantic search with high accuracy. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 915–938, Boston, MA, July 2025. USENIX Association. URL: https://www.usenix.org/conference/osdi25/presentation/zhu-jinhao.

A Noise Analysis

We prove the statements from §3.5.

Proposition 1. *Let* $\mathbf{q} \in \{-\psi, \dots, \psi-1\}^l$ *be the query embedding of the client and let* $(\mathbf{v}_i)_{i\in[N]} \in \{-\psi, \dots, \psi-1\}^l$ *be the embedding vectors in cluster* j'*. If the plaintext modulus* $p > 2l\psi^2$ *and* $\mathsf{ct}_{\mathsf{scores}}$ *decrypts correctly, then* Response *(Algorithm 6) outputs an RLWE ciphertext encrypting the polynomial whose* i^{th} *coefficient is* $\langle \mathbf{q}, \mathbf{v}_i \rangle$.

Proof. By the construction of the cluster polynomials in §3.3, the ith coefficient of $\sum_{k\in[l]} \mathfrak{p}_k \cdot \mathbf{q}[k]$ equals $\langle \mathbf{q}, \mathbf{v}_i \rangle$ for the ith embedding $\mathbf{v}_i$ of cluster j. Since RLWE encryption is linearly homomorphic (Equation (1)), $\mathsf{ct}_{\mathsf{j}} = \mathsf{sum}((\mathfrak{p}_k \cdot \mathsf{ct}_{\mathbf{q}[k]})_{k\in[l]})$ encrypts exactly this polynomial. By the definition of $\mathsf{RLWE}\times\mathsf{RGSW}$, the ciphertext $\mathsf{RLWE}\times\mathsf{RGSW}(\mathsf{ct}_{\mathsf{j}}, \mathsf{ct}_{\mathbf{u}[j]})$ encrypts the product of that polynomial with $\mathbf{u}_{j'}[j]$, which is 0 for all $j \neq j'$ and 1 for $j = j'$. Summing over all $j \in [c]$ yields an encryption of the score polynomial of cluster j'. Each score is a sum of l products of integers of magnitude at most ψ, so $|\langle \mathbf{q}, \mathbf{v}_i \rangle| \leq l\psi^2 < p/2$ prevents wrap-around. Hence, every score is retrieved correctly. ■

Lemma 1 (Adapted from [16])**.** *Let* ct *be an LWE sample with dimension n, modulus q, and error variance* σ^2*. Let* ct_{R} *be the RLWE ciphertext obtained by applying* LWEtoRLWE *to* ct*. The variance* σ_{R}^2 *is bounded as*

$$\sigma_{\mathsf{R}}^2 \leq \frac{1}{12} n^3 B_1^2 d_1 \sigma^2,$$

for a decomposition base B_1 *and* $d_1 = \lfloor \log_{B_1} q \rfloor + 1$.

Proof. To analyze the output noise variance, we first take a look at what LWEtoRLWE does, where EvalAuto is an automorphism evaluated on ct:

Algorithm 9: LWEtoRLWE [16]

$\mathsf{ct}' \leftarrow \mathsf{ct}$;
for $k \leftarrow 1$ **to** $\log n$ **do**
  $\mathsf{ct}' \leftarrow \mathsf{ct}' + \mathsf{EvalAuto}(\mathsf{ct}')$;
return ct';

We start by analyzing the noise due to EvalAuto, which uses the key-switching (KS) algorithm from [16]. Let $\mathbf{g}_1$ be a gadget vector for KS with $\mathbf{g}_1 = (1, B_1, B_1^2, \ldots, B_1^{d_1-1})$, where $d_1 = \lfloor \log_{B_1} q \rfloor + 1$. Let $\mathbf{e} \xleftarrow{\chi_e} \mathbb{Z}_q^{d_1 \times n}$ be a vector of error polynomials, where each coefficient of each polynomial is sampled from $\mathcal{X}_e$. The additional noise introduced during KS is

$$e_{ks} = \langle \mathbf{g}_1^{-1}(\mathbf{a}), \mathbf{e} \rangle,$$

where $\mathbf{a}$ is the random part of the LWE sample. The vector $\mathbf{g}_1^{-1}(\mathbf{a})$ is of length d_1 and its values are in the range $[-\lfloor (B_1-1)/2 \rfloor; \lfloor B_1/2 \rfloor]$. Thus, we can upper-bound the output variance of EvalAuto as

$$\sigma_{B_1}^2 \leq \frac{1}{4} n B_1^2 d_1 \sigma^2, \tag{3}$$

where the factor n is due to polynomial multiplication.

Unlike [16], we bound each entry of $\mathbf{g}_1^{-1}(\mathbf{a})$ by its worst-case magnitude $B_1/2$. We do not assume it is uniformly distributed over the digit range. This gives a variance factor of $1/4$ instead of $1/12$. Hence, Equation (3) is a strictly stronger, assumption-free bound on the KS digit term than the heuristic-average bound in [16].

The total error after a KS operation is $e + e_{ks}$, where e is the error of ct with variance σ^2 and e_{ks} is the additional noise due to KS with variance σ_{ks}^2. Hence, we can write the error at step k as

$$e_k = e_{k-1} + \tau(e_{k-1}) + e_{ks},$$

where $\tau(\cdot)$ is the automorphism applied by EvalAuto. The initial error of the LWE sample e is only in the constant term. So it does not get affected by automorphisms. Thus, it adds up n times due to LWEtoRLWE. But since LWEtoRLWE preprocesses the ciphertexts by multiplying them with n^{-1}, this operation will not incur noise growth on e. Its contribution to the final noise will stay the same, with variance σ^2.

With the initial error out of the way, we can now consider $e_0 = 0$. At each step k, the variance increases according to

$$\sigma_k^2 = 2^2 \sigma_{k-1}^2 + \sigma_{B_1}^2,$$

because automorphisms preserve the constant term. So the error in the constant term gets added onto itself, quadrupling the variance.

The additional variance caused by $\log n$ key-switches is then

$$\left(1 + 4 + \cdots + 4^{\log(n)-1}\right) \sigma_{B_1}^2 = \frac{n^2-1}{3} \sigma_{B_1}^2.$$

Thus, the variance of the error after applying LWEtoRLWE to a fresh LWE sample is

$$\sigma_{\mathsf{R}}^2 = \sigma^2 + \frac{n^2-1}{3} \sigma_{B_1}^2.$$

Plugging in Equation (3), we get

$$\begin{aligned} \sigma_{\mathsf{R}}^2 &\leq \sigma^2 + \frac{n^2-1}{3} \cdot \frac{1}{4} n B_1^2 d_1 \sigma^2 \\ &< \frac{1}{12} n^3 B_1^2 d_1 \sigma^2, \end{aligned}$$

where the last inequality holds as long as $B_1^2 d_1 \geq 12$. ■

Lemma 2 (Adapted from [26])**.** *Let* ct' *be a fresh LWE′ ciphertext with dimension* n*, modulus* q*, error variance* σ^2*, ternary secret key, decomposition base* B_2 *and* $d_2 = \lfloor \log_{B_2} q \rfloor + 1$*. Let* ct_{G} *be the RGSW ciphertext obtained by applying* LWE′toRGSW *to* ct'*, where* LWEtoRLWE *uses decomposition base* B_1 *and* $d_1 = \lfloor \log_{B_1} q \rfloor + 1$*. The variance* σ_{G}^2 *of* ct_{G} *is bounded as*

$$\sigma_{\mathsf{G}}^2 \leq \frac{1}{4} n \left(\frac{1}{3} n^3 B_1^2 d_1 + B_2^2 d_2 \right) \sigma^2.$$

Proof. Recall from Equation (2) that an RGSW ciphertext has two parts $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s} \cdot \mathfrak{m})$ and $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{m})$. The algorithm LWE′toRGSW works on these two parts to obtain an RGSW ciphertext as follows. To obtain the second part $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{m})$ from ct', the algorithm calls LWEtoRLWE for all LWE ciphertexts that make up the LWE′ ciphertext ct'. The resulting variance for this second part is therefore simply σ_{R}^2. To obtain the first part $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s} \cdot \mathfrak{m})$, the algorithm uses $\mathbf{K}_2 = \mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s}^2)$ published by the client on all components of $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{m})$ to obtain $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s} \cdot \mathfrak{m})$.

Fix $j \in [d_2]$ and let $(\mathfrak{a}, \mathfrak{b}) = \mathrm{RLWE}(\mathfrak{s}, B_2^j \mathfrak{m})$. Write $\mathbf{K}_2[i] = \mathrm{RLWE}(\mathfrak{s}, B_2^i \mathfrak{s}^2)$ for its ith component. The server computes

$$\begin{aligned} \langle \mathbf{g}_2^{-1}(\mathfrak{a}), \mathbf{K}_2 \rangle + (\mathfrak{b}, 0) &= \sum_{i=0}^{d_2-1} \mathfrak{a}_i \cdot \mathbf{K}_2[i] + (\mathfrak{b}, 0) \\ &= \sum_{i=0}^{d_2-1} \mathfrak{a}_i \cdot \mathbf{K}_2[i] + (\mathfrak{b}, -\mathfrak{b} \cdot \mathfrak{s} + \mathfrak{b} \cdot \mathfrak{s}) \\ &= \mathrm{RLWE}\left(\mathfrak{s}, \sum_{i=0}^{d_2-1} B_2^i \mathfrak{a}_i \cdot \mathfrak{s}^2 + \mathfrak{b} \cdot \mathfrak{s}\right) \\ &= \mathrm{RLWE}(\mathfrak{s}, (\mathfrak{a} \cdot \mathfrak{s} + \mathfrak{b}) \cdot \mathfrak{s}) \\ &= \mathrm{RLWE}(\mathfrak{s}, (B_2^j \mathfrak{m} + \mathfrak{e}_{\mathsf{R}}) \cdot \mathfrak{s}). \end{aligned}$$

where $\mathfrak{e}_{\mathsf{R}}$ is the noise due to LWEtoRLWE. Since LWEtoRLWE operates over a power-of-two cyclotomic ring, the coefficient and canonical embeddings coincide up to a constant factor. We therefore bound the multiplicative noise growth from $\mathfrak{e}_{\mathsf{R}} \cdot \mathfrak{s}$ directly in the coefficient domain. Then,

the error variance of the obtained $\mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s}\cdot\mathfrak{m})$ is upper bounded by

$$\begin{aligned}\sigma_{\mathsf{G}}^2 &\le n\sigma_{\mathsf{R}}^2 + \frac{1}{4}nB_2^2 d_2\sigma^2 \\ &\le \frac{1}{12}n^4 B_1^2 d_1\sigma^2 + \frac{1}{4}nB_2^2 d_2\sigma^2 \\ &= \frac{1}{4}n\left(\frac{1}{3}n^3 B_1^2 d_1 + B_2^2 d_2\right)\sigma^2,\end{aligned}$$

where the first term is due to the additive $\mathfrak{e}_{\mathsf{R}}\cdot\mathfrak{s}$, and the second term is due to gadget decomposition with base B_2. Since $\sigma_{\mathsf{G}}^2 \ge n\sigma_{\mathsf{R}}^2 \ge \sigma_{\mathsf{R}}^2$, the variance of the second RGSW component (equal to σ_{R}^2) is also bounded by σ_{G}^2. Hence, σ_{G}^2 is a valid bound on the noise variance of the RGSW ciphertext. ■

Lemma 3. *Let ct_{R} be an RLWE ciphertext with degree N, modulus q and error variance σ_{R}^2. Let $\mathfrak{p}\in\mathcal{R}_p$ be a plaintext polynomial with $\mathrm{Coefs}(\mathfrak{p})=\mathbf{p}$ and $|\mathbf{p}[i]|\le\psi$. Then the variance resulting from $\mathfrak{p}\cdot\mathsf{ct}_{\mathsf{R}}$ is bounded by $\psi^2 N\sigma_{\mathsf{R}}^2$.*

Proof. The coefficients of $\mathfrak{p}$ are from the set $\{-\psi,\dots,\psi-1\}$. Each coefficient of $\mathfrak{p}$ has magnitude at most ψ, scaling the variance of each convolution term by a factor of ψ^2. The factor N is due to polynomial multiplication. ■

Lemma 4 (Adapted from [16, 26])**.** *Let ct_{R} be an RLWE ciphertext with degree N, modulus q and error variance σ_{R}^2. Let ct_{G} be an RGSW ciphertext with degree N, modulus q, error variance σ_{G}^2, decomposition base B_2 and $d_2=\lfloor\log_{B_2} q\rfloor+1$. Then the variance of $\mathsf{ct}_\times=\mathsf{RLWE}\times\mathsf{RGSW}(\mathsf{ct}_{\mathsf{R}},\mathsf{ct}_{\mathsf{G}})$, namely $\sigma_\times^2$, is bounded as*

$$\sigma_\times^2 \le \frac{1}{2}Nd_2B_2^2\sigma_{\mathsf{G}}^2 + \sigma_{\mathsf{R}}^2.$$

Proof. Let $\mathsf{ct}_{\mathsf{R}} = (\mathfrak{a}, \mathfrak{b} = -\mathfrak{a}\cdot\mathfrak{s}+\mathfrak{e}+\mathfrak{m})$ and let $\mathsf{ct}_{\mathsf{G}} = (\mathsf{ct}_{\mathsf{G}1}, \mathsf{ct}_{\mathsf{G}2})$, where $\mathsf{ct}_{\mathsf{G}1} = \mathrm{RLWE}'(\mathfrak{s}, \mathfrak{s}\cdot\mathfrak{m}_{\mathsf{G}})$ and $\mathsf{ct}_{\mathsf{G}2} = \mathrm{RLWE}'(\mathfrak{s}, \mathfrak{m}_{\mathsf{G}})$, so that

$$\mathsf{ct}_{\mathsf{G}1}[i] = \mathrm{RLWE}(\mathfrak{s}, B_2^i\mathfrak{s}\cdot\mathfrak{m}_{\mathsf{G}}), \quad \mathsf{ct}_{\mathsf{G}2}[i] = \mathrm{RLWE}(\mathfrak{s}, B_2^i\mathfrak{m}_{\mathsf{G}}).$$

Let $\mathbf{g}_2 = (1, B_2, \dots, B_2^{d_2-1})$ and let $(\mathfrak{a}_i)_i = \mathbf{g}_2^{-1}(\mathfrak{a})$ and $(\mathfrak{b}_i)_i = \mathbf{g}_2^{-1}(\mathfrak{b})$, where $\mathbf{g}_2^{-1}$ is applied coefficient-wise and all sums are over $i\in[d_2]$. From [26], it follows that

$$\begin{aligned}\mathsf{RLWE}\times\mathsf{RGSW}(\mathsf{ct}_{\mathsf{R}},\mathsf{ct}_{\mathsf{G}}) &\coloneqq \langle\mathbf{g}_2^{-1}(\mathfrak{a}), \mathsf{ct}_{\mathsf{G}1}\rangle + \langle\mathbf{g}_2^{-1}(\mathfrak{b}), \mathsf{ct}_{\mathsf{G}2}\rangle \\ &= \sum_i \mathfrak{a}_i\cdot\mathsf{ct}_{\mathsf{G}1}[i] + \sum_i \mathfrak{b}_i\cdot\mathsf{ct}_{\mathsf{G}2}[i] \\ &= \sum_i \mathrm{RLWE}(\mathfrak{s}, B_2^i\mathfrak{a}_i\cdot\mathfrak{s}\cdot\mathfrak{m}_{\mathsf{G}}) \\ &\quad + \sum_i \mathrm{RLWE}(\mathfrak{s}, B_2^i\mathfrak{b}_i\cdot\mathfrak{m}_{\mathsf{G}}) \\ &= \mathrm{RLWE}(\mathfrak{s}, (\mathfrak{a}\cdot\mathfrak{s}+\mathfrak{b})\cdot\mathfrak{m}_{\mathsf{G}}) \\ &= \mathrm{RLWE}(\mathfrak{s}, (\mathfrak{m}+\mathfrak{e})\cdot\mathfrak{m}_{\mathsf{G}}).\end{aligned}$$

The result encrypts an additive error of $\mathfrak{e}\cdot\mathfrak{m}_{\mathsf{G}}$, in addition to noise added due to gadget decomposition. Hence,

$$\begin{aligned}\sigma_\times^2 &\le \frac{1}{4}Nd_2B_2^2(2\sigma_{\mathsf{G}}^2) + \mathfrak{m}_{\mathsf{G}}^2\cdot\sigma_{\mathsf{R}}^2 \\ &\le \frac{1}{2}Nd_2B_2^2\sigma_{\mathsf{G}}^2 + \sigma_{\mathsf{R}}^2.\end{aligned}$$

The last inequality holds since the message $\mathfrak{m}_{\mathsf{G}}$ is either an all-zero vector or a constant-term polynomial 1. ■

Theorem 1. *Let $\mathsf{ct}_{\mathbf{q}}$ and $\mathsf{ct}_{\mathbf{u}}$ be fresh LWE and LWE′ ciphertexts with dimension n, modulus q and error variance σ^2, respectively. Let $\mathsf{ct}_{\mathbf{q}}$ and $\mathsf{ct}_{\mathbf{u}}$ be the two ciphertext inputs to* Response *(Algorithm 6). The error variance of the output ciphertext $\mathsf{ct}_{\mathsf{scores}}$ is then bounded as*

$$\sigma_{\mathsf{scores}}^2 < cn^2\left(n^3B_1^2B_2^2d_1d_2 + B_2^4d_2^2 + n^2l\psi^2B_1^2d_1\right)\sigma^2,$$

where c is the number of clusters, B_1 and B_2 are decomposition bases, $d_1=\lfloor\log_{B_1} q\rfloor+1$, $d_2=\lfloor\log_{B_2} q\rfloor+1$, and ψ is an upper bound on the absolute value of the plaintext polynomials' coefficients.

Proof. From Lemma 1, each $\mathsf{ct}_{\mathbf{q}[i]}$ has variance σ_{R}^2. When it gets multiplied with a plaintext polynomial $\mathfrak{p}_i$, the resulting ciphertext has variance $\psi^2N\sigma_{\mathsf{R}}^2$. Summing over l such ciphertexts $\forall i\in[l]$ results in $\mathsf{ct}_{\mathbf{r}_j}$ with variance $l\psi^2N\sigma_{\mathsf{R}}^2$, due to Lemma 3. From Lemma 2, $\mathsf{ct}_{\mathbf{u}[j]}$ has variance σ_{G}^2.

The RLWE ciphertext $\mathsf{ct}_{\mathbf{r}_j}$ and the RGSW ciphertext $\mathsf{ct}_{\mathbf{u}[j]}$ are now the inputs to the $\mathsf{RLWE}\times\mathsf{RGSW}$ algorithm. From Lemma 4, the output variance $\sigma_\times^2$ is then bounded as

$$\sigma_\times^2 \le \frac{1}{2}Nd_2B_2^2\sigma_{\mathsf{G}}^2 + l\psi^2N\sigma_{\mathsf{R}}^2.$$

We add up c ciphertexts to obtain

$$\sigma_{\mathsf{scores}}^2 \le c\sigma_\times^2.$$

Plugging in $\sigma_\times^2$, σ_{G}^2 and σ_{R}^2, we obtain

$$\begin{aligned}\sigma_{\mathsf{scores}}^2 &\le c\left(\frac{1}{2}Nd_2B_2^2\sigma_{\mathsf{G}}^2 + l\psi^2N\sigma_{\mathsf{R}}^2\right) \\ &\le c\left(\frac{1}{2}Nd_2B_2^2\left[\frac{1}{4}n\left(\frac{1}{3}n^3B_1^2d_1 + B_2^2d_2\right)\sigma^2\right]\right. \\ &\quad \left. + l\psi^2N\sigma_{\mathsf{R}}^2\right) \\ &\le \frac{1}{24}cNn^4B_1^2B_2^2d_1d_2\sigma^2 + \frac{1}{8}cNnB_2^4d_2^2\sigma^2 \\ &\quad + \frac{1}{12}cNn^3l\psi^2B_1^2d_1\sigma^2 \\ &< cNn\left(n^3B_1^2B_2^2d_1d_2 + B_2^4d_2^2 + n^2l\psi^2B_1^2d_1\right)\sigma^2.\end{aligned}$$

We replace $N=n$ due to LWEtoRLWE. ■

Remark 3. Notice that the bound in Theorem 1 can be tightened further. (1) The bound from Lemma 2 treats both parts of the RGSW ciphertext as if they had the same variance σ_G^2. Actually, one of the parts has variance $\sigma_R^2 < \sigma_G^2$. (2) The bound in Lemma 4 assumes that the RGSW ciphertexts encrypt a constant-term polynomial 1, whereas all but one of our RGSW ciphertexts encrypt an all-zero polynomial. Incorporating these changes would result in a tighter bound. In this paper, we state the bound above, but we use the tightened bound for our parameter selection.

Corollary 1. *Let* $\Delta = \lfloor q/p \rceil$ *and* $\mathsf{ct}_{\mathsf{scores}}$ *be bounded as in Theorem 1. Then,* $\mathsf{ct}_{\mathsf{scores}}$ *decrypts correctly except with probability at most*

$$N \cdot \operatorname{erfc}\left(\frac{\Delta}{2\sqrt{2}\sigma_{\mathsf{scores}}}\right).$$

Taking the optimizations in Sections 3.2.3 and 3.2.4 into account, for r CRT component rings and $\lceil D/(Nc) \rceil$ *splits, the quantity above scales up by* $r\lceil D/(Nc) \rceil$.

Proof. Decryption of an RLWE ciphertext succeeds as long as every error coefficient satisfies $|\mathfrak{e}_i| < \Delta/2$ (§2.2). For a Gaussian error coefficient of variance $\sigma_{\mathsf{scores}}^2$,

$$\Pr\left[|\mathfrak{e}_i| \geq \Delta/2\right] = \operatorname{erfc}\left(\frac{\Delta}{2\sqrt{2}\sigma_{\mathsf{scores}}}\right),$$

and a union bound over the N coefficients of one response ciphertext, and then over the $r\lceil D/(nc) \rceil$ response ciphertexts the client decrypts, gives the result. ■

B Evaluation

We evaluate Mímir's scoring retrieval performance over the parameter sets shown in Table 3. Mímir can be tailored to different use cases through different parameter choices. The main trade-off is between search quality, query latency and communication cost.

Semantic embedding model. Mímir supports any semantic embedding model. The decision on which model suits the private search protocol best may be influenced by the following attributes: *embedding length, token length, similarity measure, model size (in parameters), model speed, and model quality*. Table 4 gives an overview of the properties of different models we considered for Mímir. The quality of the models is compared via MRR@100, calculated from the first 100 queries over a subset of 5100 documents of the MS MARCO dataset. Besides using the distilbert model for a fair comparison against Tiptoe and PACMANN, we instantiate Mímir with the state-of-the-art lightweight embedding model *bge-small-en-v1.5* trained with cosine similarity (§2), for the best trade-off between fast query generation, low communication cost, and high search quality.

Unless explicitly stated otherwise, the following MRR@100 results are obtained using the dot product of the distilbert model [76].

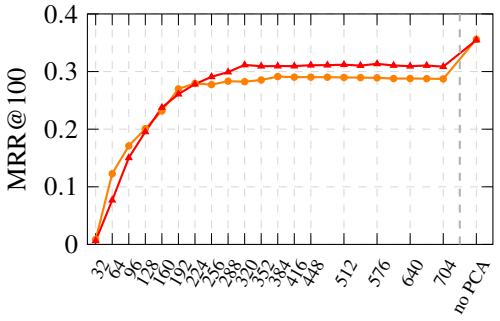


Figure 5: Search quality for PCA reduced embedding dimension l over 100 and 500 sample queries.

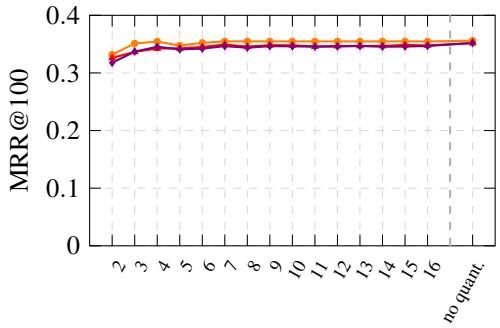


Figure 6: Search quality for the integer precision ρ over 100, 500 and 1000 sample queries).

Embedding dimension l. One parameter that influences the quality of search results is the embedding dimension l, which we reduce via principal component analysis (PCA). Shorter embedding vectors reduce communication and computation costs, but they also lose information, leading to lower search quality. Figure 5 shows the quality of the brute-force search (without clustering, using `float32` vectors), based on the MRR@100 for the first 100 and 500 queries of the MS MARCO dataset, comparing different embedding vector dimensions.

Embedding precision ρ. We follow [36] in converting the embedding vectors from `float32` data types to low-precision integers. This allows our cryptographic scheme to use a smaller plaintext modulus p and leads to smaller ciphertexts in general. Figure 6 shows the quality of a brute-force search (without clustering or PCA), based on the MRR@100 for the first 100 queries (500 queries in red, 1000 in purple) of the MS MARCO dataset, comparing different integer precisions. We observe that quantization leads to information loss only at very small precisions; in some cases it even increases MRR@100.

Table 3: Parameters for Mímir I–IV and Mímir-bge.

Parameter	Mímir I	Mímir II	Mímir III	Mímir IV	Mímir-bge
Security parameter, $n = N$	2048	2048	4096	2048	2048
Standard deviation of the error, σ	3.2	3.2	3.2	3.2	3.2
Ciphertext modulus q	53 bits	53 bits	58 bits	53 bits	53 bits
Security level, λ	128.8	128.8	248.5	128.8	128.8
Embedding vector length, l	192	192	192	192	256
Embedding precision ρ	5 bits	5 bits	5 bits	5 bits	3 bits
Plaintext modulus, p	98305	98305	98305	98305	8193
Number of CRT component rings, r	2	2	1	2	1
Component ring moduli, p_R	$\{314, 315\}$	$\{314, 315\}$	98305	$\{314, 315\}$	8193
Number of clusters c	393	4	1	10	7
Decomposition base B_1	3	3	5	5	5
LWEtoRLWE decomposition degree d_1	34	34	25	23	23
Decomposition base B_2	17	179	3	98	3
LWE′toRGSW decomposition degree d_2	13	7	37	8	34

Table 4: Comparison of embedding models. MRR@100 is measured over 5100 documents and 100 queries of MS MARCO. The small documentset is the reason for the comparatively high MRR@100. Time is the total encoding time for all queries and documents. Params lists the number of model parameters to download.

Model	Similarity	Dim	MRR@100	Time (s)	Params
bge-small-en-v1.5	dot-product	384	0.9422	97.0	33M
	cosine	384	0.9422		
all-MiniLM-L6-v2	dot-product	384	0.9022	81.3	22M
	cosine	384	0.9022		
msmarco-distilbert-base-tas-b	dot-product	768	0.6483	199.3	66M
	cosine	768	0.5594		

B.1 Communication Breakdown

One-time costs. Before issuing any query, the client uploads three objects that the server reuses across all future queries. A $O(\lambda)$-bit seed lets the server locally compute the uniformly random components $\mathbf{A}$ and $\mathbf{a}$ of the client's scheme-switching keys $\mathbf{K}_1$ and $\mathbf{K}_2$. The key $\mathbf{K}_1$ lets the server perform LWEtoRLWE on the client's ciphertexts. The key $\mathbf{K}_2$, alongside $\mathbf{K}_1$, lets the server perform LWE′toRGSW.

Online costs. Each query consists of an LWE encryption $\mathsf{ct}_\mathbf{q}$ of the query embedding and an LWE′ encryption $\mathsf{ct}_\mathbf{u}$ of the cluster-selection vector. Since the plaintext space is decomposed into r component rings, one ciphertext per component ring is sent for the query embedding. The ciphertext $\mathsf{ct}_\mathbf{u}$ only encrypts zeros and ones and does not need to be decomposed. For the uniformly random parts of the ciphertexts, a single $O(\lambda)$-bit seed is transmitted instead of explicit ring elements. The server can generate all the uniformly random parts of the received ciphertexts by expanding this seed. On the download side, the server represents the database embeddings over the same r component rings and splits each cluster of size D/c into splits of size n. The client downloads one response ciphertext per component ring and per split.

Table 5: Communication costs of Mímir's scoring retrieval.

Item	Size (bits)
One-time Upload	
$\mathsf{seed}^\dagger$	$O(\lambda)$
Scheme-switching key $\mathbf{K}_1$	$\log(N) N d_1 \lceil \log q \rceil$
Scheme-switching key $\mathbf{K}_2$	$N d_2 \lceil \log q \rceil$
Upload - Online	
$\mathsf{seed}^\dagger$	$O(\lambda)$
Embedding query $\mathsf{ct}_\mathbf{q}$	$rl \lceil \log q \rceil$
Cluster query $\mathsf{ct}_\mathbf{u}$	$d_2 c \lceil \log q \rceil$
Download - Online	
Response $\mathsf{ct}_{\mathsf{scores}}$	$r \lceil D/(Nc) \rceil 2N \lceil \log q \rceil$

B.2 Computation Breakdown

Core primitives. Three algorithms recur throughout the online phase. We cost each once here, and write C_X for the multiplication cost of algorithm X.

LWEtoRLWE runs the algorithm depicted in [16]. This performs $\log N$ KeySwitch calls. Each call costs $(d_1 + 1)(N/2)\log N + 2d_1 N$ multiplications. Summed over $\log N$ calls, $C_{\mathsf{LWEtoRLWE}} = N \log N [((d_1 + 1)/2)\log N + 2d_1]$.

LWE′toRGSW is a black box combining two steps. The first step, LWE′toRLWE′, applies LWEtoRLWE to each of

the d_2 LWE ciphertexts making up the LWE′ ciphertext. This costs $d_2 C_{\mathsf{LWEtoRLWE}}$. The second step, RLWE′toRGSW, uses the scheme-switching technique of [26]. For each of the d_2 RLWE ring elements, the component is converted to the coefficient domain, gadget-decomposed into d_2 short ring elements, and each ring element converted back to the NTT domain before the pointwise multiplications. This costs $d_2 N\left[((d_2+1)/2)\log N + 2d_2\right]$ multiplications. Summing both steps gives $C_{\mathsf{LWE'toRGSW}}$ as in Table 6.

RLWE×RGSW multiplies an RLWE ciphertext by an RGSW ciphertext. Both components of the RLWE ciphertext are gadget-decomposed, requiring two conversions to the coefficient domain. Each decomposition yields d_2 short ring elements, each requiring reconversion to the NTT domain, for $2d_2$ conversions in total. The resulting $4d_2$ digits are each pointwise-multiplied against the RGSW ciphertext. This costs $C_{\mathsf{RLWE\times RGSW}} = N\left[(d_2+1)\log N + 4d_2\right]$.

Table 6: Computational costs of core primitives.

Algorithm	Cost (multiplications)
$C_{\mathsf{LWEtoRLWE}}$	$N\log N\left[((d_1+1)/2)\log N + 2d_1\right]$
$C_{\mathsf{LWE'toRGSW}}$	$d_2 C_{\mathsf{LWEtoRLWE}} +$ $d_2 N\left[((d_2+1)/2)\log N + 2d_2\right]$
$C_{\mathsf{RLWE\times RGSW}}$	$N\left[(d_2+1)\log N + 4d_2\right]$

One-time costs (client). The client generates a secret key $\mathbf{s}$. It converts $\mathbf{s}$ into the NTT domain. This costs $N/2\log N$ multiplications. The result, $\mathsf{NTT}(\mathfrak{s})$, is reused in every step below. The client then runs $\mathsf{LWEtoRLWE-KeyGen}$ and $\mathsf{RLWEtoRGSW-KeyGen}$, producing the scheme-switching keys $\mathbf{K}_1$ and $\mathbf{K}_2$. Both are computed and stored in the NTT domain.

$\mathsf{LWEtoRLWE-KeyGen}$ samples d_1 ring elements. These can be sampled directly in the NTT domain. Each is multiplied by an automorphism of $\mathsf{NTT}(\mathfrak{s})$. This costs $d_1 N$ multiplications. To each product we add $\mathfrak{s}$ times $\mathbf{g}_1$, the gadget vector. Each entry of $\mathbf{g}_1$ plaintext-multiplies $\mathfrak{s}$. This costs another $d_1 N$ multiplications. The client also samples d_1 error ring elements. It converts them to the NTT domain. This costs $d_1(N/2)\log N$ multiplications. In total, $\mathbf{K}_1$ costs $d_1 N\left[(1/2)\log N + 2\right]$ multiplications.

$\mathsf{RLWEtoRGSW-KeyGen}$ computes $\mathbf{K}_2 = \mathrm{RLWE}'(\mathfrak{s},\mathfrak{s}^2)$. It first computes $\mathfrak{s}^2$ from $\mathsf{NTT}(\mathfrak{s})$. This costs N multiplications. The ciphertext consists of d_2 RLWE ciphertexts. Each has the form $\mathfrak{a}_i\cdot\mathfrak{s} + \mathfrak{e}_i + B_2^i\mathfrak{s}^2$. For each $i\in[d_2]$, the client computes $\mathfrak{a}_i\cdot\mathfrak{s}$. This costs N multiplications. It samples $\mathfrak{e}_i$ and converts it to the NTT domain. This costs $N/2\log N$ multiplications. It computes $B_2^i\mathfrak{s}^2$. This costs another N multiplications. Summed over d_2 ciphertexts and added to the $\mathfrak{s}^2$ step, $\mathbf{K}_2$ costs $d_2 N\left[(1/2)\log N + 2\right] + N$ multiplications.

One-time costs (server). The server NTT-transforms the vector embeddings. The embeddings are arranged into polynomials first. Each cluster spans r component rings. Each cluster is split into $\lceil D/(cN)\rceil$ splits. Each split contains l polynomials. In total, this costs $crl\lceil D/(cN)\rceil (N/2)\log N$ multiplications.

Online costs (client). The client encrypts rl embedding messages. Each encryption costs n multiplications. This step costs rln multiplications in total. The client also encrypts an LWE′ ciphertext for the cluster-selection vector. This ciphertext has c entries and gadget length d_2. Each entry costs n multiplications. This step costs $d_2 cn$ multiplications in total.

The client then decrypts the server's response. The response consists of $r\lceil D/(Nc)\rceil$ RLWE ciphertexts. Each decryption costs N multiplications. This step costs $r\lceil D/(Nc)\rceil N$ multiplications in total. Each decrypted message is then converted back to the coefficient domain. This costs $r\lceil D/(Nc)\rceil (N/2)\log N$ multiplications in total.

Online costs (server). The server performs four steps to answer a query, each built from the primitives of Table 6.

First, the server switches the query embedding into RLWE form. There are rl embedding ciphertexts, one per component ring and embedding coordinate. Each switch is one LWEtoRLWE call.

Second, the server converts the cluster selection ciphertext into RGSW. There is one cluster selection ciphertext per cluster. Each conversion is one LWE′toRGSW call.

Third, the server multiplies the switched embedding query against the plaintext database. This happens once per component ring, cluster, embedding coordinate, and split. Both operands are already NTT-resident, and plaintext multiplication requires no gadget decomposition, so each multiplication costs a plain pointwise product: $2N$. This step costs $2Nrcl\lceil D/(Nc)\rceil$.

Fourth, the server selects the relevant cluster by multiplying against the RGSW-encrypted cluster indicator. This happens once per component ring, cluster, and split. Each multiplication is one RLWE×RGSW call, so this step costs $rc\lceil D/(Nc)\rceil C_{\mathsf{RLWE\times RGSW}}$. Table 8 summarizes these costs.

B.3 Latency

Table 7 shows the computation time for all parameter sets, each evaluated under three execution schemes: fully sequential processing, multithreaded processing over 32 threads on a single machine, and multithreaded and distributed processing across 10 servers. Total latency is given as the sum of query generation, server computation, and client decryption; it excludes any network or inter-process communication overhead, as all schemes – including the distributed configuration – were simulated on a single machine without network communication. Figures for all parameter sets are averages over 50 (seq.) and 100 (MT and dist.) runs. Measurements were taken on an AMD Ryzen 9 7950X3D 16-core / 32-thread processor (observed core clock ≈ 5.05 GHz). All results including the median and standard deviation can be found in [1].

Table 7: Per-query latency breakdown (in milliseconds) of all parameter sets for sequential (Seq.), multithreaded (MT), and distributed (Dist.) execution.

Parameter set	N	Clusters	Exec.	Client registration	Query generation	Server computation breakdown						Client decryption	Total latency
						Server computation	Query unpacking	RLWE switching†	RGSW switching†	Scoring calc.†	Response packing		
Mímir I	2048	393	Seq.	**77**	**600**	**67839**	59	4037	55728	8016	0	**0**	**68440**
			MT	**73**	**633**	**3600**	60	209	2833	495	3	**1**	**4233**
			Dist.	**74**	**627**	**806**	60	49	642	52	3	**1**	**1433**
Mímir II	2048	4	Seq.	**74**	**44**	**11216**	4	4038	300	6853	20	**55**	**11315**
			MT	**72**	**52**	**765**	5	213	95	439	13	**56**	**873**
			Dist.	**73**	**50**	**218**	5	49	93	58	14	**54**	**323**
Mímir III	4096	1	Seq.	**134**	**86**	**10484**	5	3640	779	6022	39	**113**	**10683**
			MT	**132**	**98**	**1432**	5	195	812	373	48	**128**	**1658**
			Dist.	**131**	**88**	**1279**	5	31	815	381	47	**106**	**1474**
Mímir IV	2048	10	Seq.	**51**	**50**	**10642**	5	2960	636	7033	8	**22**	**10714**
			MT	**50**	**58**	**692**	6	151	85	443	7	**24**	**774**
			Dist.	**49**	**54**	**177**	5	35	79	48	9	**22**	**253**
Mímir-bge	2048	7	Seq.	**57**	**53**	**10530**	5	1960	2054	6505	6	**16**	**10599**
			MT	**54**	**61**	**840**	6	103	335	387	9	**18**	**919**
			Dist.	**55**	**58**	**414**	6	15	323	61	9	**15**	**487**

†For the distributed execution, RLWE ciphertext switching, RGSW ciphertext switching and scoring calculations reflect the workload processed by a single machine's share of the clusters. The benchmarks assume a cluster of 10 machines. For $c * r < 10$ our distributed implementation does not reach its optimal performance.

Table 8: Computational costs of Mímir's scoring retrieval.

Item	**Cost (multiplications)**
One-time Client	
Secret key $\mathsf{NTT}(\mathfrak{s})$	$(N/2)\log N$
Scheme-switching key $\mathbf{K}_1$	$d_1 N\left[(1/2)\log N+2\right]$
Scheme-switching key $\mathbf{K}_2$	$d_2 N\left[(1/2)\log N+2\right]+N$
One-time Server	
NTT	$crl\lceil D/(Nc)\rceil(N/2)\log N$
Client - Online	
$\mathrm{LWE}(\mathbf{s},\mathbf{q})$	rln
$\mathrm{LWE}'(\mathbf{s},\mathbf{u}_{j'})$	$d_2 cn$
Decrypting $\mathsf{ct}_{\mathsf{scores}}$	$r\lceil D/(Nc)\rceil N$
INTT	$r\lceil D/(Nc)\rceil(N/2)\log N$
Server - Online	
$\mathsf{LWEtoRLWE}(\mathsf{ct}_{\mathbf{q}},\mathbf{K}_1)$	$rl\,C_{\mathsf{LWEtoRLWE}}$
$\mathsf{LWE'toRGSW}(\mathsf{ct}_{\mathbf{u}},\mathbf{K}_1,\mathbf{K}_2)$	$c\,C_{\mathsf{LWE'toRGSW}}$
$\mathsf{PtxtMult}$	$2Nrcl\,\lceil D/(Nc)\rceil$
$\mathsf{RLWE}\times\mathsf{RGSW}$	$rc\,\lceil D/(Nc)\rceil\,C_{\mathsf{RLWE}\times\mathsf{RGSW}}$